



TÜRKISCH-DEUTSCHE UNIVERSITÄT
FAKULTÄT FÜR INGENIEURWISSENSCHAFTEN
STUDIENRICHTUNG MECHATRONIK
STUDIENJAHR 2024-2025
SOMMERSEMESTER

Entwurf und Entwicklung der autonomen Drohne

Bachelorarbeit

Vorbereitet von

200507024 Ahmet Yüksel ERKURT

200507017 Yankı Bora AYMAN

Betreuer

Assist. Prof. Dr. Ali Can KAYA

Istanbul, 2025

Danksagung

An dieser Stelle möchten wir unseren besonderen Dank an Herrn Dr. Ali Can Kaya richten, der uns während der gesamten Entstehung dieser Bachelorarbeit mit außergewöhnlichem Engagement und fachlicher Expertise begleitet hat. Seine wertvollen Hinweise, seine analytische Denkweise und seine motivierende Art haben nicht nur zur wissenschaftlichen Qualität dieser Arbeit beigetragen, sondern uns auch persönlich in unserer Entwicklung als Ingenieure gestärkt.

Durch seine geduldige Betreuung und seine Bereitschaft, auch außerhalb regulärer Zeiten für Rückfragen zur Verfügung zu stehen, hat er maßgeblich dazu beigetragen, Herausforderungen konstruktiv zu bewältigen und kreative Lösungsansätze zu finden. Besonders in komplexen technischen Fragestellungen war seine Unterstützung für uns von unschätzbarem Wert.

Für seine kontinuierliche Ermutigung, seine klaren Rückmeldungen und das Vertrauen, das er in uns gesetzt hat, möchten wir ihm unseren aufrichtigen Dank aussprechen.

Kurzfassung

Ziel dieser Bachelorarbeit ist die Entwicklung und Implementierung eines autonomen Überwachungsdrohnensystems, das in der Lage ist, definierte Gebiete selbstständig zu überwachen, potenzielle Objekte zu erkennen und relevante Informationen in Echtzeit zu verarbeiten. Dafür wurde ein Quadrocopter mit einer modularen Hardware-Architektur aufgebaut, bestehend aus einem Speedybee F405 V4 Flugcontroller, einem Raspberry Pi als Begleitcomputer sowie einem GPS-Modul und einer Kameraeinheit.

Die Flugsteuerung basiert auf dem Open-Source-Framework ArduPilot, wobei die Kommunikation zwischen Flugcontroller und Begleitrechner über das MAVLink-Protokoll erfolgt. Zur Objekterkennung wurde das neuronale Netzwerk YOLOv4 Tiny auf dem Raspberry Pi implementiert, welches in der Lage ist, Personen und Fahrzeuge in Echtzeit zu identifizieren. Die erfassten Daten werden lokal in einer SQLite-Datenbank gespeichert und können über eine Bodenstation weiterverarbeitet werden.

Zur Validierung des Systems wurden verschiedene Flugtests durchgeführt. Dabei konnte gezeigt werden, dass der entwickelte Prototyp zuverlässig entlang vordefinierter Wegpunkte navigieren und gleichzeitig Objekterkennung mit angemessener Genauigkeit durchführen kann.

Diese Arbeit leistet einen Beitrag zur Entwicklung kosteneffizienter, autonomer UAV-Lösungen für den Einsatz in Bereichen wie Sicherheit, Katastrophenschutz und Umweltmonitoring.

Schlüsselwörter:

Autonome Drohne, Flugsteuerung, YOLOv4 Tiny, MAVLink, ArduPilot, Speedybee F405 V4, Objekterkennung, Echtzeitverarbeitung, UAV, SQLite, Wegpunktnavigation

Acknowledgements

At this point, we would like to express our sincere gratitude to Dr. Ali Can Kaya, who supported us throughout the development of this bachelor thesis with exceptional dedication and technical expertise. His valuable feedback, analytical mindset, and motivating attitude not only enhanced the scientific quality of our work but also contributed significantly to our personal development as engineers.

Thanks to his patient guidance and his willingness to assist us even beyond regular hours, we were able to overcome technical challenges and develop creative solutions. Especially when dealing with complex system-related questions, his support proved to be invaluable.

We are truly grateful for his continuous encouragement, his constructive criticism, and the trust he placed in us throughout this project.

Abstract

The aim of this bachelor thesis is the development and implementation of an autonomous surveillance drone system capable of independently monitoring defined areas, detecting potential objects, and processing relevant data in real time. For this purpose, a quadcopter was constructed using a modular hardware architecture consisting of a Speedybee F405 V4 flight controller, a Raspberry Pi as companion computer, a GPS module, and a camera unit.

The flight control is managed through the open-source ArduPilot framework, while communication between the components is established via the MAVLink protocol. For object detection, the lightweight neural network YOLOv4 Tiny was implemented on the Raspberry Pi, enabling real-time recognition of people and vehicles. Detected data is stored locally using an SQLite database and can be accessed via a ground control station.

Multiple flight tests were conducted to validate the system. The results demonstrated that the developed prototype is capable of reliable waypoint navigation while providing accurate and fast object detection.

This work contributes to the advancement of cost-effective autonomous UAV solutions, which can be applied in fields such as security monitoring, disaster response, and environmental observation.

Keywords:

Autonomous drone, flight control, YOLOv4 Tiny, MAVLink, ArduPilot, Speedybee F405 V4, object detection, real-time processing, UAV, SQLite, waypoint navigation

Inhaltsverzeichnis

Abbildungsverzeichnis	5
1. Einleitung.....	7
1.1 Motivation.....	7
1.2 Problemstellung	8
1.3 Ziel der Arbeit.....	8
2. Stand der Technik	10
2.1 Autonome Drohnen: Anwendungsbereiche und Entwicklungen.....	10
2.2 Überblick über existierende Systeme.....	10
2.3 Forschung zu Bildverarbeitung und Flugsteuerung	11
3. Systementwurf und Hardwarearchitektur	12
3.1 Überblick über den Systemaufbau	12
3.2 Der Flugcontroller Speedybee F405 V4	13
3.3 Elektronischer Drehzahlregler (ESC)	14
3.4 Energieversorgung durch Li-Ion-Akkupack	15
3.5 Der Begleitcomputer: Raspberry Pi 3B+	16
3.6 GPS-Modul (einziger externer Sensor).....	18
3.7 Motorenwahl	19
3.8 Propellerwahl	20
3.8.1 Geometrie: Zweiblatt gegenüber Dreiblatt-Propellern.....	20
3.8.2 Materialwahl: Polycarbonat gegenüber Nylon.....	21
3.8.3 Vergleich verschiedener Propellergeometrien	21
3.8.4 Begründung für die konkrete Wahl	22
3.8.5 Hersteller: ETHiX	22
3.9 Funkverbindung über Radiolink (TBS Tango 2)	22
3.10 Mechanischer Aufbau und 3D-gedruckte Komponenten.....	24
3.10.1 GPS-Modul-Halterung	24
3.10.2 Halterung für den Begleitcomputer (Raspberry Pi)	25
3.10.3 Kamera-Halterung	25
3.10.4 Druckprozess und Materialwahl.....	26
3.11 Rahmenwahl, Systemmontage und Verkabelung	26
3.11.1 Eigenschaften des Rahmens	27
3.11.2 Systemmontage	28
4.2 MAVLink-Protokoll.....	31
4.3 Datenverarbeitung mit SQLite	32
5.1 Technische Grundlagen von YOLOv4 Tiny	34
5.2 Modellarchitektur und Konvertierung nach ONNX	36

5.3 Integration mit OpenCV auf dem Raspberry Pi.....	37
5.4 Ereignismodellierung und Datenverknüpfung	38
5.5 Datenbankdesign mit SQLite	40
5.6 Validierung der Objekterkennung und Grenzen	42
5.7 Visualisierung der Daten.....	43
6. Flugplanung und Durchführung von Tests	45
6.2 Testumgebung und Szenarien	47
6.3 Auswertung der Testergebnisse	48
6.3.1 Navigation und Flugverhalten	48
6.3.2 Objekterkennung	48
6.3.3 Systemstabilität und Ressourcennutzung	48
7. Zusammenfassung und Ausblick.....	50
7.1 Erkenntnisse und Beitrag der Arbeit.....	50
7.2 Potenziale für Weiterentwicklung und zukünftige Anwendungen	50
8. Literaturverzeichnis	51

Abbildungsverzeichnis

Abbildung 1: Aufblick über den Systemaufbau	12
Abbildung 2: Speedybee F405 V4 [59]	13
Abbildung 3: Speedybee 4in1 ESC 55 Amper [60].....	14
Abbildung 4: 4S Lithium-Ionen-Akkupack	15
Abbildung 5: Entladerate (C-Rate) Graph	15
Abbildung 6: Raspberry Pi 3B+[57].....	16
Abbildung 7: M10Q 250 GPS-Modul [58].....	18
Abbildung 8: Velox 2207 1750 KV bürstenlose Gleichstrommotoren [56].....	19
Abbildung 9: Ethix P3B-Propeller [61]	20
Abbildung 10: TBS Tango 2 [62]	22
Abbildung 11: 3D-gedruckte Komponenten	24
Abbildung 12: Festigung des Gehäuses.....	25
Abbildung 13: Verkabelung der Energieabschaltung [63]	26
Abbildung 14: Verkabelung von gesamten elektrischen Komponenten	28
Abbildung 15: Benutzeroberfläche der ArduPilot-Plattform [65]	29
Abbildung 16: Schematische Darstellung der MAVLink-Verbindung [64]	31
Abbildung 17: Erstellung der Datenbank	32
Abbildung 18: YOLOv4 Tiny Funktionsprinzip [67]	34
Abbildung 19: ONNX Model Schema	36
Abbildung 20: Code von yolov4tiny.py	37
Abbildung 21: Unsere Beispiel Datenbank	40
Abbildung 22: Ein Beispiel für Objekterkennung von Autos [69].....	42
Abbildung 23: Logikdiagramm	43
Abbildung 24: Code von main.py.....	44
Abbildung 25: Der gesamte Aufbau des Drohnes	45
Abbildung 26: Navigationsteuerung auf Ardupilot [68]	45

Tabellenverzeichnis

Tabelle 1: Vergleichstabelle	11
Tabelle 2: Vergleich verschiedener Propellergeometrien.....	21
Tabelle 3: Datenstruktur Schema.....	38
Tabelle 4: Event Tabelle	41

Abkürzungsverzeichnis

Abkürzung	Bedeutung
UAV	Unmanned Aerial Vehicle (Unbemanntes Luftfahrzeug)
FC	Flight Controller – in diesem Projekt: Speedybee F405 V4
RPi	Raspberry Pi – Begleitcomputer zur Bildverarbeitung
GCS	Ground Control Station – Bodenstation zur Missionskontrolle
GPS	Global Positioning System – Satellitengestützte Positionsbestimmung
IMU	Inertial Measurement Unit – Sensorpaket zur Lage- und Bewegungsmessung
ESC	Electronic Speed Controller – Motorsteuerung
MAVLink	Micro Air Vehicle Communication Protocol – Kommunikationsprotokoll zwischen FC und Companion Computer
ArduPilot	Open-Source-Flugsteuerungssoftware für autonome Missionen
YOLOv4-Tiny	You Only Look Once v4 – Leichtgewichtige Objekterkennung mit Deep Learning
CNN	Convolutional Neural Network – Neuronales Netz zur Bildverarbeitung
FPS	Frames Per Second – Bildverarbeitungsrate
PWM	Pulse Width Modulation – Steuerungssignal für Motoren
UART	Universal Asynchronous Receiver-Transmitter – Serielle Kommunikationsschnittstelle
SQLite	Lightweight SQL Database – Lokale Datenspeicherung auf dem RPi
GND	Ground – Masseanschluss in elektronischen Schaltungen
GNSS	Global Navigation Satellite System – Globale Satellitennavigation (z. B. GPS, GLONASS, Galileo)

1. Einleitung

Die rasante Entwicklung von unbemannten Luftfahrzeugen (UAVs) hat in den letzten Jahren zu einem Paradigmenwechsel in zahlreichen zivilen, industriellen und sicherheitsrelevanten Bereichen geführt. Drohnen finden heute Anwendung in der Landwirtschaft, der Umweltüberwachung, der Logistik, bei Such- und Rettungseinsätzen sowie im Bereich der Gebäude- und Gebietssicherheit. Besonders hervorzuheben ist dabei der technologische Trend in Richtung Autonomie, der die Abhängigkeit von menschlicher Steuerung reduziert und den Betrieb in schwer zugänglichen oder risikoreichen Umgebungen ermöglicht.

Während kommerzielle Systeme meist leistungsstark und vielseitig einsetzbar sind, sind sie oft auch kostspielig, komplex in der Handhabung und auf spezialisierte Infrastruktur angewiesen – insbesondere in Form von Bodenstationen, Cloud-Systemen oder proprietären Softwarelösungen. Diese Hürden erschweren den Zugang zu autonomen UAV-Systemen für Bildungseinrichtungen, kleine Organisationen und private Anwender.

Diese Arbeit verfolgt daher das Ziel, ein kostengünstiges, modulares und vollständig autonomes Drohnensystem zu entwickeln, das ohne klassische Bodenstation arbeitet. Stattdessen erfolgt die Steuerung und Kommunikation direkt über ein Radiolink-System (TBS Tango 2), wodurch zusätzliche Infrastruktur entfällt und die Systemarchitektur bewusst schlank gehalten wird. Als Flugcontroller wird der Speedybee F405 V4 eingesetzt – eine leistungsfähige und erschwingliche Alternative zu weit verbreiteten Steuerungen wie Pixhawk. Für die autonome Bildverarbeitung und Entscheidungsfindung dient ein Raspberry Pi, auf dem ein leichtgewichtiges neuronales Netzwerk (YOLOv4 Tiny) zur Objekterkennung in Echtzeit läuft.

Der besondere Fokus liegt auf der Integration verschiedener Subsysteme: Flugsteuerung mit ArduPilot, Kommunikation über MAVLink, visuelle Objekterkennung mit künstlicher Intelligenz, und Datenverarbeitung mittels SQLite – alles in einem einzigen, autonom agierenden System. Durch diese Integration soll die Drohne in der Lage sein, vordefinierte Routen autonom abzufliegen, Objekte wie Personen oder Fahrzeuge selbstständig zu erkennen, und erkannte Ereignisse zu dokumentieren – etwa zur Identifikation von unbefugtem Betreten (Trespassing) in sicherheitsrelevanten Bereichen.

Diese Arbeit ist somit nicht nur ein ingenieurwissenschaftliches Projekt, sondern auch ein Beitrag zur Zugänglichkeit und Demokratisierung moderner UAV-Technologie. Durch Offenheit, niedrige Kosten und geringe Bedienhürden kann dieses System als Grundlage für zukünftige Entwicklungen, insbesondere in der Lehre und praxisnahen Forschung, dienen.

1.1 Motivation

Der Zugang zu Drohnentechnologie ist trotz ihrer weitreichenden Einsatzmöglichkeiten oftmals durch hohe Kosten und technisches Vorwissen beschränkt. Viele existierende Systeme erfordern spezialisierte Bodenstationen, teure Komponenten und fortgeschrittene Kenntnisse in Flugsteuerung und Programmierung. Dies stellt insbesondere für Bildungseinrichtungen, Hobbyanwender und kleine Organisationen eine erhebliche Einstiegshürde dar.

Diese Arbeit verfolgt daher das Ziel, zu zeigen, dass auch mit einfachen, günstigen und frei verfügbaren Mitteln ein leistungsfähiges autonomes UAV-System realisiert werden kann. Durch den bewussten Verzicht auf eine klassische GCS und die Integration eines Radiolink-Systems (TBS Tango 2) wird eine besonders kompakte und zugängliche Lösung geschaffen.

Zudem erlaubt die Autonomie des Systems, dass es auch ohne Vorwissen im Bereich UAV-Steuerung eingesetzt werden kann. Die automatische Navigation sowie die auf

künstlicher Intelligenz basierende Objekterkennung tragen maßgeblich dazu bei, dass selbst unerfahrene Nutzer mit der Drohne effektiv arbeiten können. Damit leistet dieses Projekt einen Beitrag zur Demokratisierung von Drohnentechnologie.

1.2 Problemstellung

Trotz der rasanten technologischen Entwicklungen im Bereich der unbemannten Luftfahrzeuge besteht weiterhin eine große Lücke zwischen professionellen, kostenintensiven Systemen und benutzerfreundlichen, preiswerten Lösungen für spezifische Anwendungsfälle. Viele existierende UAV-Systeme erfordern eine umfangreiche Infrastruktur wie Bodenstationen, komplexe Flugplanungstools und fortgeschrittene Nutzerschulung. Dies macht ihren Einsatz für kleine Organisationen, Bildungseinrichtungen oder private Sicherheitsanwendungen oft unpraktisch.

Ein zentrales Problem besteht in der Vereinbarkeit von Autonomie, Kompaktheit und Funktionalität. Insbesondere bei sicherheitsrelevanten Anwendungen – wie dem Schutz von privaten oder industriellen Arealen – wäre es von Vorteil, wenn eine Drohne selbstständig ein festgelegtes Gebiet überwachen und potenzielle unerlaubte Eindringlinge (Trespassing) erkennen könnte, ohne dass ein menschlicher Pilot oder eine externe Infrastruktur erforderlich ist.

Konventionelle Systeme mit Live-Videoübertragung, manueller Steuerung und externer Analyse erfordern oft permanente Überwachung, was sowohl personelle als auch technische Ressourcen bindet. Eine autonom agierende Drohne, die in der Lage ist, Objekte zu identifizieren, Positionen zu loggen und Eindringlinge direkt vor Ort zu erkennen, könnte einen erheblichen Mehrwert bieten.

Zusätzlich erschwert die technische Integration verschiedener Komponenten wie Flugregelung, Kommunikation, Bildverarbeitung und Datenmanagement in einem ressourcenschonenden Gesamtsystem die Umsetzung solcher Funktionen. Die Herausforderung liegt daher in der Entwicklung eines robusten, intelligenten und einfach bedienbaren Drohnensystems, das sowohl autonom als auch anpassbar ist – und das zu einem Bruchteil der Kosten kommerzieller Lösungen.

1.3 Ziel der Arbeit

Ziel dieser Bachelorarbeit ist die Konzeption, Umsetzung und Validierung eines vollständig autonomen, kosteneffizienten und leicht bedienbaren Drohnensystems, das ohne klassische Bodenstation arbeitet und auch für sicherheitskritische Aufgaben wie das Erkennen unbefugten Eindringens (Trespassing) in einem definierten Gebiet eingesetzt werden kann.

Im Zentrum steht die Entwicklung eines modularen UAV-Prototyps, der mithilfe eines Speedybee F405 V4 Flight Controllers und eines Raspberry Pi als Onboard-Computer eine autonome Navigation entlang vordefinierter Wegpunkte ermöglicht. Das System soll in der Lage sein, in Echtzeit Objekte wie Personen oder Fahrzeuge zu erkennen, ihre Position zu speichern und kritische Ereignisse (z. B. Grenzverletzungen) lokal zu verarbeiten. Dabei wird auf externe Infrastruktur wie eine Ground Control Station vollständig verzichtet; die Kommunikation erfolgt ausschließlich über ein Radiolink-System (TBS Tango 2).

Konkret verfolgt die Arbeit folgende technische und funktionale Ziele:

- Hardwareseitiger Aufbau eines kompakten und leichten Quadropters mit optimierter Energieversorgung,
- Implementierung eines Echtzeit-Objekterkennungssystems (YOLOv4 Tiny) auf dem Raspberry Pi,
- Anbindung des Begleitcomputers an den Flight Controller über das MAVLink-Protokoll zur Synchronisation von Sensor- und Steuerdaten,
- Integration eines lokalen Datenmanagements über SQLite zur strukturierten Speicherung erkannter Objekte und Ereignisse,
- Validierung durch reale Flugtests mit unterschiedlichen Testumgebungen und Szenarien,
- Minimierung von Benutzerinteraktion zur Ermöglichung eines Plug-and-Play-Betriebs, auch für unerfahrene Anwender.

Darüber hinaus verfolgt die Arbeit ein übergeordnetes Ziel: den Nachweis der Realisierbarkeit eines autonomen, intelligenten Drohnensystems auf Basis frei verfügbarer, günstiger Komponenten. Durch die Veröffentlichung der Konzepte und der Implementierungsdetails kann das System als Open-Source-Vorlage für ähnliche Projekte dienen – insbesondere in Bildungskontexten oder für Sicherheitsanwendungen mit begrenztem Budget.

2. Stand der Technik

Die Entwicklung autonomer UAV-Systeme hat in den letzten Jahren erhebliche Fortschritte gemacht. Besonders im Bereich der Low-Cost-Drohnen werden zunehmend offene Plattformen, modulare Hardwarekomponenten und auf künstlicher Intelligenz basierende Algorithmen eingesetzt. In diesem Kapitel wird ein Überblick über den aktuellen Stand der Technik gegeben – mit Fokus auf Technologien, die in diesem Projekt Anwendung finden.

2.1 Autonome Drohnen: Anwendungsbereiche und Entwicklungen

Autonome Drohnen haben sich zu einem unverzichtbaren Werkzeug in zahlreichen zivilen, industriellen und sicherheitsrelevanten Bereichen entwickelt. Ihre Fähigkeit, Aufgaben ohne direkte menschliche Steuerung autonom auszuführen, macht sie besonders attraktiv für Anwendungen, bei denen Effizienz, Sicherheit und Reaktionsgeschwindigkeit entscheidend sind [8].

Zivile und industrielle Einsatzgebiete: Im zivilen Bereich kommen autonome UAVs u. a. in der Präzisionslandwirtschaft, Umweltüberwachung, im Katastrophenschutz, bei Infrastrukturinspektionen und in der öffentlichen Sicherheit zum Einsatz. Beispielsweise ermöglichen sie eine zielgerichtete Applikation von Düngemitteln sowie die Zustandserfassung von Vegetation [2]. Auch bei Such- und Rettungseinsätzen bieten sie Vorteile durch thermische Sensorik und den Zugang zu schwer erreichbaren Gebieten.

Sicherheitsanwendungen: Im sicherheitstechnischen Kontext dienen Drohnen zunehmend der Gebietsüberwachung. Mithilfe von Kameramodulen können sie definierte Routen autonom abfliegen und Eindringlinge detektieren [7]. Gerade für großflächige oder sensible Zonen stellt dies eine kosteneffiziente Alternative zu festen Sensorstrukturen dar.

Technologischer Fortschritt: Fortschritte in der Sensorintegration, Edge Computing und KI-Optimierung haben eine Echtzeitverarbeitung visueller Daten an Bord ermöglicht. Embedded-Systeme wie der Raspberry Pi in Kombination mit CNNs wie YOLOv4 Tiny erlauben eine latenzarme Bildverarbeitung [6]. Gleichzeitig bieten Flight Controller wie der Speedybee F405 V4 eine ausreichende Leistungsfähigkeit für autonome Missionen zu geringen Kosten [1].

Gesellschaftliche und regulatorische Aspekte: Rechtliche Rahmenbedingungen, Datenschutz und gesellschaftliche Akzeptanz sind wesentliche Faktoren für den Einsatz autonomer Drohnen in öffentlichen Räumen. Drohnen müssen betriebssicher, datenschutzkonform und ethisch vertretbar einsetzbar sein [8].

2.2 Überblick über existierende Systeme

Systeme mit GCS-basierter Kontrolle: Viele Systeme nutzen Ground Control Stations (GCS) zur Flugplanung und Steuerung, z. B. in der Arbeit von Moran Yustas [7], die ArduPilot mit einer GCS kombiniert. Diese Systeme benötigen jedoch komplexe Infrastruktur und technisches Know-how. Im Gegensatz dazu verzichtet das hier vorgestellte System komplett auf eine GCS und nutzt stattdessen ein direktes Radiolink-Modul (TBS Tango 2).

AIoT-gestützte Systeme mit ArduPilot: Die Studie "AI Wings" von Liu et al. [5] zeigt die Möglichkeiten von AIoT-Drohnen mit Jetson Nano und YOLOv4. Diese Systeme sind leistungsfähig, aber hardwareintensiv. Unser System setzt auf Raspberry Pi mit YOLOv4 Tiny

und demonstriert, dass auch mit kostengünstiger Hardware ein funktionales Echtzeit-AI-System realisierbar ist.

GNSS-denied Navigation: Kumar et al. [4] beschreiben ein System, das LIDAR- und Kameradaten zur Navigation ohne GNSS nutzt. Aufgrund des Fokus auf offene Flächen mit GNSS-Verfügbarkeit nutzt unser System klassisch GNSS in Kombination mit einer IMU.

Systeme zur Objekterkennung: Die Arbeit von Mendoza et al. [6] zeigt eine OpenCV-basierte Objekterkennung mit Onboard-Verarbeitung. Unser System verwendet YOLOv4 Tiny als ressourcenschonende Weiterentwicklung mit höherer Genauigkeit und Echtzeitfähigkeit.

2.3 Forschung zu Bildverarbeitung und Flugsteuerung

Vergleichstabelle:

Merkmal	Bestehende Systeme	Unser Projekt
Flugcontroller	Pixhawk/PX4	Speedybee F405 V4
GCS erforderlich	Ja	Nein
Objekterkennung	OpenCV-basiert	YOLOv4 Tiny mit CNN
KI-Hardware	Jetson, GPU	Raspberry Pi
Datenspeicherung	Extern/Cloud	Lokal via SQLite
Telemetrie	GCS-gebunden	Radiolink (TBS Tango 2)

Tabelle 1: Vergleichstabelle

Bildverarbeitung: Leichtgewichtige neuronale Netze wie YOLOv4 Tiny ermöglichen Echtzeiterkennung auf Embedded-Systemen. Liu et al. [5] demonstrieren die Leistungsfähigkeit auf Jetson Nano. Mendoza et al. [6] zeigen, dass durch Modelloptimierung auch ein Raspberry Pi fähig ist, verarbeitbare Ergebnisse zu liefern. Die Architektur folgt dem Konzept der Edge Intelligence zur Reduktion von Latenz [8].

Flugsteuerung: ArduPilot als Open-Source-Software ermöglicht autonome Flugmodi, die über MAVLink mit externer Peripherie kommunizieren. MAVLink ermöglicht dabei effiziente Datenübertragung bei geringem Overhead [3]. Audronis [1] zeigt die Integration von ArduPilot mit Companion Computern. Unser System übernimmt diese Architektur auf einer kostengünstigen Plattform.

Sensorfusion: Präzise Flugregelung erfordert Sensorfusion (GNSS, IMU, Barometer, ggf. visuelle SLAM). Systeme wie das von Kumar et al. [4] zeigen das Potenzial visueller Navigation in GNSS-denied Umgebungen. Unser System beschränkt sich bewusst auf GNSS + IMU, da die Testbedingungen eine klassische Navigation erlauben.

3. Systementwurf und Hardwarearchitektur

3.1 Überblick über den Systemaufbau

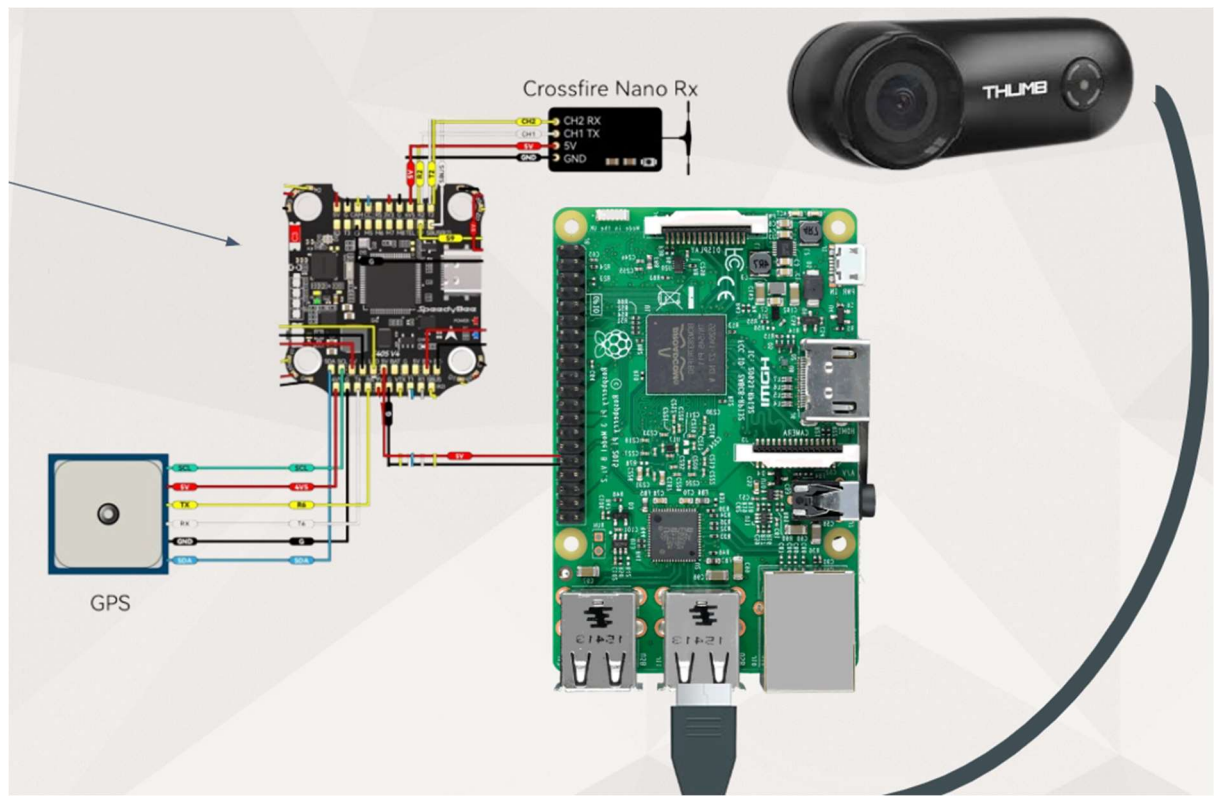


Abbildung 1: Aufblick über den Systemaufbau

Das entworfene System basiert auf einer autonomen Multikopterplattform, die für die Erkennung von Personen und Fahrzeugen in abgegrenzten Sicherheitszonen konzipiert wurde. Ziel ist es, eine vollständig autonome Drohne zu realisieren, die selbstständig startet, vordefinierte Wegpunkte abfliegt, Objekte erkennt und identifizierte Ereignisse lokal speichert, bevor sie automatisch zum Startpunkt zurückkehrt.

Das System besteht aus zwei logisch getrennten Subsystemen:

- Flugsteuerungssystem (Flight Controller – FC):

Die Flugstabilisierung, Navigation und Missionsplanung basieren auf dem Open-Source-Autopilot-System ArduPilot, welches auf einem Speedybee F405 V4 Flugcontroller implementiert ist. Der FC ist für Waypoint-Navigation, Stabilisierung, Sensorfusion und Fail-Safe-Mechanismen verantwortlich. Die gesamte Flugtrajektorie wird autonom anhand vordefinierter Missionsdaten ausgeführt.

- Bildverarbeitungseinheit (Companion Computer – CC):

Für die Objekterkennung wird ein Raspberry Pi 3B+ als Begleitrechner eingesetzt, der unabhängig vom FC arbeitet. Die Kommunikation erfolgt über eine serielle MAVLink-Verbindung, über welche ausschließlich GPS-Positionsdaten vom FC empfangen werden. Der Raspberry Pi analysiert den Videostream der RunCam Thumb Kamera mithilfe von YOLOv4-Tiny (implementiert über OpenCV + ONNX) und speichert erkannte Objekte (z. B. Person, Auto) samt Zeit- und Ortsangabe in einer lokalen SQLite-Datenbank.

- Kommunikation & Steuerung:

Die Fernsteuerung und Telemetrie erfolgt über das TBS Crossfire-System mit dem TBS Tango 2 Sender. Diese Verbindung erlaubt MAVLink-kompatible Übertragungen über große Distanzen (20+ km), wobei die manuelle Steuerung im Notfall möglich bleibt, der Flug aber vollständig autonom geplant und ausgeführt wird.

- Mechanik und Stromversorgung:

Die Drohne basiert auf einem TBS Source Two 5" Frame aus Carbonfaser. Die Energieversorgung erfolgt durch einen 4S Li-Ion Akku (Sony VTC6, 2800 mAh). Ein integrierter 4-in-1 ESC regelt die Velox 2207 1750kv Brushless-Motoren. 3D-gedruckte Halterungen aus PLA fixieren GPS-Modul, Kamera und Begleitrechner sicher auf dem Frame.

3.2 Der Flugcontroller Speedybee F405 V4

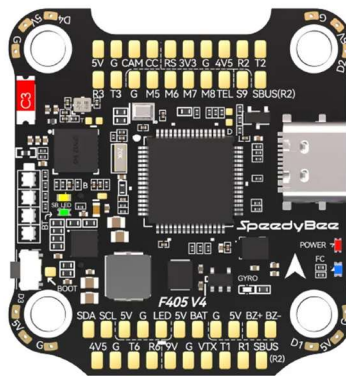


Abbildung 2: Speedybee F405 V4 [59]

Im Zentrum der autonomen Steuerung steht der Speedybee F405 V4 Flugcontroller, der die komplette Regelung und Navigation der Drohne übernimmt. Als Herzstück des Systems ist er verantwortlich für Flugstabilität, Sensordatenfusion, Positionsbestimmung und die Ausführung autonomer Missionen.

Der Controller basiert auf dem leistungsfähigen STM32F405 Mikrocontroller, der sich aufgrund seiner Rechenkapazität und Kompatibilität mit ArduPilot ideal für einfache autonome Anwendungen eignet [9]. In Kombination mit dem integrierten Beschleunigungs- und Lagesensor (IMU), dem Kompass sowie dem GNSS-Modul kann der Flugcontroller präzise Positions- und Lagedaten in Echtzeit verarbeiten [3].

Ein entscheidender Vorteil dieses Moduls ist seine weite Verbreitung in der Maker- und FPV-Community. Dadurch stehen umfangreiche Online-Dokumentationen, Tutorials und Troubleshooting-Hilfen zur Verfügung, was insbesondere im Rahmen eines studentischen Projekts die Implementierung erheblich erleichtert [1].

Die autonome Missionsausführung basiert auf der Nutzung des ArduPilot-Autopiloten, welcher auf dem Speedybee F405 V4 konfiguriert wurde. Die Missionen werden durch definierte Waypoints realisiert, bei denen das Flugverhalten (z. B. Geschwindigkeit, Zielhöhe) detailliert festgelegt werden kann. Die Start- und Landesequenzen erfolgen ebenfalls vollautomatisch [1].

Zusätzlich wurde der Flugcontroller in einem integrierten Stack mit einem 4-in-1 ESC-Modul verbaut, was die Verkabelung vereinfacht und das Gewicht reduziert (siehe Abschnitt 3.3). Die Kommunikation mit externen Komponenten, insbesondere dem Companion Computer (siehe Abschnitt 3.5), erfolgt über eine serielle UART-Schnittstelle, über die MAVLink-Daten übertragen werden [3].

Die Kombination aus einfacher Konfiguration, starker Community-Unterstützung und ArduPilot-Kompatibilität macht den Speedybee F405 V4 zu einer kostengünstigen und leistungsfähigen Lösung für autonome UAV-Anwendungen [9].

3.3 Elektronischer Drehzahlregler (ESC)



Abbildung 3: Speedybee 4in1 ESC 55 Ampere [60]

Zur Ansteuerung der vier Brushless-Motoren kommt ein integrierter 4-in-1 ESC der Marke Speedybee (55A BLS) zum Einsatz, der direkt mit dem Flight Controller (Speedybee F405 V4) in einem kompakten 30x30 mm Stack kombiniert ist. Diese Bauweise ermöglicht eine platzsparende Montage und reduziert den Verdrahtungsaufwand erheblich, was sich insbesondere in kleineren Drohnenkonstruktionen als vorteilhaft erweist [10].

Der ESC ist in der Lage, einen kontinuierlichen Strom von 55 Ampere pro Kanal zu liefern, was für die eingesetzten Motoren (Velox 2207 mit 1750 kV) mehr als ausreichend ist. Für kurzzeitige Lastspitzen ist die Einheit ebenfalls ausgelegt, was die Betriebssicherheit auch bei schnellen Manövern oder unter hoher Last gewährleistet.

Technisch basiert das Modul auf einem BLHeli_S-Firmware-Profil, welches für zuverlässige Signalansteuerung sorgt und eine gleichmäßige Drehzahlregelung der Motoren erlaubt [11]. Die Anbindung erfolgt über ein standardisiertes PWM-Protokoll direkt vom Flugcontroller aus, der über die ArduPilot-Software die Geschwindigkeit und Laufrichtung der Motoren in Echtzeit bestimmt.

Durch die enge Integration mit dem Flugcontroller ergeben sich folgende Vorteile:

- Plug-and-Play-Kompatibilität
- Reduzierung der elektrischen Störungen durch kurze Verbindungen
- Geringeres Gewicht und kompakter Aufbau
- Vereinfachte Fehlersuche und Konfiguration

Die Auswahl eines integrierten ESC-Stacks war eine bewusste Entscheidung zur Erhöhung der Systemstabilität und der Reduzierung des mechanischen Aufwands bei der Montage. Diese Komponente stellt somit ein zentrales Element im elektrischen Design der Drohne dar und unterstützt aktiv die präzise Regelung des Flugverhaltens.

3.4 Energieversorgung durch Li-Ion-Akkupack



Abbildung 4: 4S Lithium-Ionen-Akkupack

Für die Stromversorgung des gesamten Systems wird ein 4S Lithium-Ionen-Akkupack auf Basis von Sony VTC6 (18650) Zellen verwendet. Der Akku liefert eine Gesamtspannung von 16.8 V (voll geladen) bei einer Kapazität von 2800 mAh pro Zelle und ist speziell für Anwendungen mit hohem Energiebedarf geeignet [12].

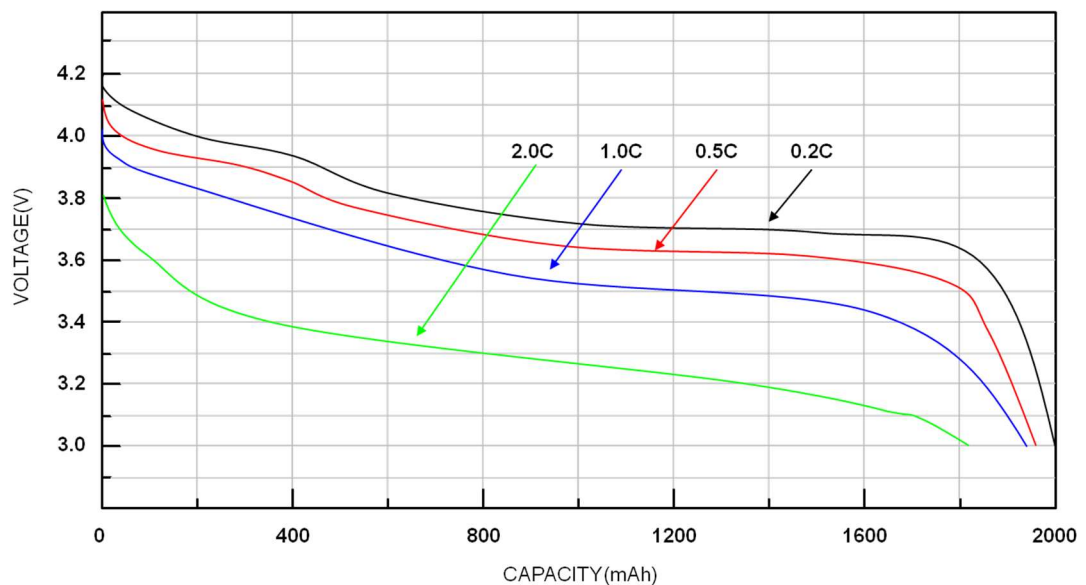


Abbildung 5: Entladerate (C-Rate) Graph

Im Vergleich zu herkömmlichen Lithium-Polymer-Akkus (LiPo) bietet dieser Akkutyp eine höhere Energiedichte, was längere Flugzeiten ermöglicht – ein entscheidender Vorteil für autonome Missionen [13]. Trotz der niedrigeren Entladerate (C-Rate) im Vergleich zu LiPo-Akkus ist die Konfiguration ausreichend, um den Leistungsbedarf der Velox 2207 1750kV Brushless-Motoren und des übrigen Systems zuverlässig zu decken.

Anschlüsse & Verkabelung:

- XT60-Stecker: Wird zur Hauptstromversorgung verwendet und ist für bis zu 180 A Spitzenstrom sowie 60 A kontinuierlich ausgelegt.
- Balancer-Kabel: Ermöglicht das gleichmäßige Laden der einzelnen Zellen und minimiert Spannungsunterschiede zwischen diesen.
- 12 AWG-Silikonkabel: Für die Stromleitung zwischen Akku, ESC und Verteilerplatine verwendet; geeignet für bis zu 160 A Spitze und 25 A Dauerstrom [14].

Ladegerät: Zum Laden des Akkus wird das Modell ToolkitRC M4Q verwendet, das für Li-Ion und LiPo-Akkus konzipiert ist. Es ermöglicht präzises Balancing und ist mit Sicherheitsfunktionen wie Überhitzungs- und Überspannungsschutz ausgestattet [14].

Sicherheit und Schutz: Die Verwendung von Li-Ion-Akkus erfordert erhöhte Vorsicht bezüglich:

- Überladungsschutz
- Zellspannungsüberwachung
- Thermisches Management [13]

Dank der Kombination aus hoher Energiedichte und ausreichender Strombelastbarkeit ist dieser Akkupack ideal geeignet für das vorliegende Projekt, bei dem Flugzeit und Zuverlässigkeit gleichermaßen wichtig sind [13].

3.5 Der Begleitcomputer: Raspberry Pi 3B+

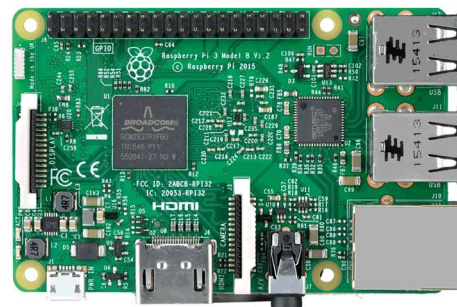


Abbildung 6: Raspberry Pi 3B+[57]

Zur Realisierung bildverarbeitungsbasierter Funktionalitäten sowie zur lokalen Ereignisprotokollierung wurde ein Raspberry Pi 3B+ als sogenannter Begleitcomputer (Companion Computer) in das Gesamtsystem integriert. Diese Komponente arbeitet vollständig unabhängig vom Flugcontroller, führt jedoch über eine serielle Schnittstelle eine minimale Datenverbindung zum Hauptsystem auf Basis des MAVLink-Protokolls. Diese Architektur verfolgt den Ansatz einer modularen Trennung zwischen Navigations- und Erkennungseinheit, wie sie auch in vergleichbaren UAV-Systemen mit Vision-Komponenten etabliert ist [2], [6].

Hardwareeigenschaften und Funktion: Der Raspberry Pi 3B+ stellt eine kostengünstige, weit verbreitete und durch zahlreiche Softwarebibliotheken gut unterstützte

Plattform dar, die sich für einfache AI-Workloads wie Echtzeit-Objekterkennung eignet. Ausgestattet mit einem Quad-Core ARM Cortex-A53 Prozessor (1,4 GHz) und 1 GB RAM ist das System für die Ausführung ressourcenschonender Deep-Learning-Modelle wie YOLOv4-Tiny ausreichend dimensioniert [6]. Die geringe Größe und das niedrige Gewicht machen ihn zudem geeignet für mobile Anwendungen im UAV-Bereich.

Die Bilddaten werden über eine per USB angebundene RunCam Thumb Kamera aufgenommen, welche Videos in einer Auflösung von 1080p bei 60 fps liefert. Dank ihres Gewichts von nur 9,8 g trägt sie nur minimal zur Gesamtmasse der Drohne bei und ermöglicht eine vibrationsarme Montage auf einer dedizierten Halterung.

Software und Datenverarbeitung: Zur Objekterkennung wird ein leichtgewichtiges neuronales Netzwerkmodell implementiert: YOLOv4-Tiny, betrieben auf Basis von OpenCV und ONNX-Runtime. Dieses Framework erlaubt die framewise Verarbeitung des Video-Streams in Echtzeit und identifiziert relevante Klassen wie „Person“ oder „Auto“. Sobald ein solches Objekt erkannt wird, wird ein Ereignisprotokoll erstellt, das folgende Informationen umfasst:

- Objekttyp
- GPS-Koordinaten (vom Flugcontroller über MAVLink)
- Zeitstempel des Erkennungszeitpunkts

Diese Daten werden in einer lokalen SQLite-Datenbank abgelegt, was die Offline-Verfügbarkeit sowie nachträgliche Analyse ermöglicht [6]. Die strukturierte Speicherung nach dem Schema (objekt_typ, zeitstempel, latitude, longitude) erlaubt eine einfache Erweiterbarkeit des Systems für weitere Objekttypen oder Sensordaten.

Kommunikationsschnittstelle: Die einzige Verbindung zum Flugcontroller (Speedybee F405 V4) besteht aus einer seriellen UART-Kommunikation, über die ausschließlich Telemetriedaten empfangen werden. Eine Rückkopplung an den Flugcontroller erfolgt nicht, um Systeminterferenzen auszuschließen. Diese unidirektionale Verbindung stellt sicher, dass der Flugregler auch bei Ausfall des Begleitcomputers autonom funktionsfähig bleibt – ein Prinzip, das in sicherheitskritischen UAV-Anwendungen empfohlen wird [15].

Stromversorgung und Integration: Die Energieversorgung erfolgt direkt über den Flugcontroller. Da der Raspberry Pi unabhängig von dessen Steuerungslogik arbeitet, sind keine synchronisierten Softwaremodule notwendig. Die Integration erfolgt mechanisch über eine zweiteilige, 3D-gedruckte Gehäuseeinheit, bestehend aus einer oberen Befestigungsplattform und einer unteren Schutzabdeckung (siehe Abschnitt 3.10.3).

Der Einsatz des Raspberry Pi als dedizierter Companion Computer zeigt, dass auch mit beschränkten Ressourcen eine autonome visuelle Erkennung realisiert werden kann – ein Konzept, das in der Forschung zunehmend Beachtung findet [5], [6].

3.6 GPS-Modul (einziger externer Sensor)



Abbildung 7: M10Q 250 GPS-Modul [58]

Die präzise Positionsbestimmung ist eine zentrale Voraussetzung für die Durchführung autonomer Flugmissionen. Im vorliegenden System übernimmt diese Aufgabe das Foxeer M10Q 250 GPS-Modul, das mit einem integrierten HMC5883-Kompass ausgestattet ist. Dieses Modul stellt die einzige externe Sensoreinheit dar und liefert essentielle Daten zur globalen Lokalisierung im Raum.

Das Modul basiert auf dem neuesten u-blox M10-Chipsatz, der sich durch besonders hohen Empfang von Satellitensignalen auszeichnet. Es unterstützt den parallelen Empfang mehrerer Systeme im Rahmen des sogenannten GNSS (Global Navigation Satellite System). GNSS ist ein Überbegriff für alle satellitengestützten Ortungssysteme, einschließlich:

- GPS (USA)
- GLONASS (Russland)
- Galileo (EU)
- BeiDou (China)

Die gleichzeitige Nutzung mehrerer GNSS-Systeme verbessert die Verfügbarkeit, Redundanz und Genauigkeit der Positionsbestimmung erheblich, insbesondere unter schwierigen Umgebungsbedingungen wie in Städten, Wäldern oder bei begrenztem Sichtfeld zum Himmel. Laut Kaplan und Hegarty (2018) kann durch Mehrsystemempfang eine Genauigkeit im Sub-Meter-Bereich erzielt werden [3].

Integration in das Gesamtsystem: Die vom GPS-Modul generierten Positionsdaten werden über ein serielles UART-Protokoll an den Speedybee F405 V4 Flugcontroller übermittelt, der diese in die ArduPilot-Navigationslogik einbindet. Gleichzeitig sendet der Flugcontroller eine Auswahl an Telemetriedaten über das MAVLink-Protokoll an den Companion Computer (Raspberry Pi), um dort zur Ereignisdokumentation genutzt zu werden. Die GPS-Koordinaten bilden damit die gemeinsame Datenbasis sowohl für die Flugsteuerung als auch für die Bildverarbeitungs- und Logikmodule.

Mechanische Anbindung: Das GPS-Modul ist auf einer speziell für das Drohnengerüst entwickelten 3D-gedruckten Halterung montiert. Die Konstruktion wurde so ausgelegt, dass das Modul oberhalb des Hauptrahmens positioniert ist und so nicht im direkten Luftstrom der Propeller liegt, um Störungen zu minimieren. Zudem gewährleistet die Halterung eine sichere Kabelführung und schützt das Modul vor physischen Beschädigungen. Eine durchdachte

Positionierung und Vibrationsentkopplung sind insbesondere für den empfindlichen Kompass essenziell, um Störungen durch elektromagnetische Einflüsse der ESCs oder Motoren zu vermeiden.

3.7 Motorenwahl



Abbildung 8: Velox 2207 1750 KV bürstenlose Gleichstrommotoren [56]

Die Auswahl geeigneter Motoren ist entscheidend für das Flugverhalten, die Energieeffizienz sowie die Nutzlastkapazität eines unbemannten Flugsystems. In diesem Projekt kommen vier Velox 2207 1750 KV bürstenlose Gleichstrommotoren (BLDC) zum Einsatz, die speziell für FPV- und Hochleistungsanwendungen ausgelegt sind [16].

Technische Eigenschaften und Auswahlkriterien: Die Bezeichnung 2207 bezieht sich auf die physikalischen Abmessungen des Motors: 22 mm Statorbreite und 7 mm Statorhöhe. Diese Konfiguration bietet ein gutes Verhältnis zwischen Drehmoment und Reaktionsgeschwindigkeit, was für präzise Steuerbefehle während autonomer Missionen besonders vorteilhaft ist.

Der KV-Wert von 1750 gibt an, wie viele Umdrehungen pro Minute (RPM) der Motor pro Volt Spannung erzeugt – im vorliegenden Fall also 1750 RPM/V. Dieser Wert deutet auf einen optimalen Kompromiss zwischen Schubkraft und Energieeffizienz bei einer 4S-Lithium-Ionen-Konfiguration (16,8 V) hin [13]. Die Motoren liefern bei Bedarf einen Spitzenstrom von bis zu 36,6 A (60 Sekunden), was ausreichende Leistungsreserven für kurzfristige Lastspitzen garantiert.

Effizienz und Kompatibilität: In Kombination mit zweiblättrigen Propellern aus Polycarbonat (siehe Abschnitt 3.8) erreichen die gewählten Motoren eine hohe Schubausbeute bei gleichzeitig geringem Stromverbrauch. Durch die Wahl eines moderaten KV-Werts wurde bewusst ein energieeffizienter Betrieb favorisiert, um die Flugzeit unter Einsatz von Lithium-Ionen-Zellen zu maximieren [13].

Die Motoren sind vollständig kompatibel mit dem eingesetzten 4in1 ESC-Modul (Speedybee F405 V4 BLS 55A), welches auf einen maximalen Dauerstrom von 55 A pro Kanal ausgelegt ist. Diese Abstimmung verhindert potenzielle thermische Überlastung und ermöglicht ein sicheres Betriebsspektrum des Antriebsstrangs [10].

Bewertung: Die Entscheidung für die Velox 2207 1750 KV Motoren basiert auf einem integrativen Ansatz aus Gewicht, Stromaufnahme, mechanischer Kompatibilität und thermischer Belastbarkeit. Die Motoren wurden so gewählt, dass sie ausreichend Reserven für anspruchsvolle Flugmanöver und Windstörungen bieten, ohne den Energieverbrauch überproportional zu steigern – ein kritisches Kriterium für die Gesamtflugzeit bei einem sicherheitskritischen UAV-Einsatz.

3.8 Propellerwahl



Abbildung 9: Ethix P3B-Propeller [61]

Die Auswahl des passenden Propellers ist von zentraler Bedeutung für die Effizienz, Kontrolle und Flugstabilität eines unbemannten Luftfahrzeugs. Im Rahmen dieser Arbeit wurden zweiblättrige Ethix P3B-Propeller aus Polycarbonat mit einem Durchmesser von 5.1 Zoll eingesetzt. Diese Wahl basiert auf einer gezielten Abwägung zwischen Energieeffizienz, struktureller Integrität, Gewicht und aerodynamischer Leistung.

3.8.1 Geometrie: Zweiblatt gegenüber Dreiblatt-Propellern

Der eingesetzte P3B-Biblade-Propeller besitzt mit 5.1 Zoll einen leicht überdurchschnittlichen Durchmesser im Vergleich zu den üblichen 5.0-Zoll-Propellern, was zu einem erhöhten maximalen Schub beiträgt [18]. Ein direkter Vergleich mit der ebenfalls verfügbaren Triblade-Variante desselben Modells zeigt einen Gewichtsunterschied von ca. 24,3 % zugunsten der Biblade-Version [20].

Obwohl Dreiblattpropeller die Luft besser „greifen“ und feinfühligere Steuerung ermöglichen, zeichnen sich Zweiblattpropeller durch:

- Geringeren Luftwiderstand,
- Weniger Energieverbrauch,
- Höhere Effizienz bei gleichbleibender Schuberzeugung,

aus – Eigenschaften, die für autonome, langandauernde Missionen besonders vorteilhaft sind.

3.8.2 Materialwahl: Polycarbonat gegenüber Nylon

Bei der Materialauswahl wurden Polycarbonat- und Nylonpropeller gegenübergestellt. Die Entscheidung fiel zugunsten von Polycarbonat, da dieses Material sich durch folgende Merkmale auszeichnet [17]:

- Höhere Steifigkeit → reduziert ungewollte Verformung bei hohen Drehzahlen,
- Weniger Vibrationen → günstig für Flugstabilität und Bildverarbeitung,
- Bessere Steuerpräzision,
- Längere Lebensdauer bei mechanischer Belastung.

Ein Nachteil besteht jedoch in der erhöhten Bruchgefahr, insbesondere bei Stürzen. Für dieses Projekt stellte dies aber keinen relevanten Nachteil dar, da Vibrationen minimiert und die Objekterkennungsleistung maximiert werden sollten [19].

3.8.3 Vergleich verschiedener Propellergeometrien

Merkmal	Zweiblatt (Bi-Blade)	Dreiblatt (Tri-Blade)	Vierblatt (Quad-Blade)
Effizienz	Hoch	Mittel	Gering
Stromverbrauch	Gering	Höher	Höchster
Maximaler Schub	Niedriger	Hoch	Sehr hoch
Steuerbarkeit	Reaktiv, begrenzt präzise	Sehr feinfühlig	Stabil, aber träger
Vibrationen	Weniger	Mittel	Sehr gering
Lautstärke	Eher leise	Mittel	Laut
Typischer Einsatzzweck	Langstrecke, Effizienz	FPV-Rennen, Agilität	Schwerlast, Spezialzwecke

Tabelle 2: Vergleich verschiedener Propellergeometrien

3.8.4 Begründung für die konkrete Wahl

Das Ziel dieses Projekts – der Entwurf einer energieeffizienten, autonomen Überwachungsdrohne mit Bildverarbeitung an Bord – erfordert ein möglichst ausgewogenes Verhältnis zwischen Leistung, Gewicht und Flugzeit. Die Kombination der Velox 2207 1750KV-Motoren (siehe Kapitel 3.7) mit den leichten und aerodynamisch effizienten Zweiblatt-Propellern erwies sich in Simulation und Tests als optimal.

Die Wahl von 5.1"-Biblade-Propellern ermöglichte:

- eine Erhöhung des Gesamtschubs ohne zusätzlichen Stromverbrauch,
- die Reduktion der Massenträgheit und somit eine effizientere Flugregelung,
- die Minimierung von Vibrationen für störungsfreie Kameraaufnahmen – ein kritischer Punkt bei der Implementierung von YOLOv4 Tiny auf dem Begleitcomputer.

3.8.5 Hersteller: ETHiX

ETHiX-Propeller sind in der FPV-Community für ihre hohe Fertigungsqualität, Vibrationsbalance und Stabilität bekannt. Im Zusammenspiel mit einem Carbonrahmen und vibrationsresistenter Elektronik bilden sie eine zuverlässige Plattform für **visuelle Erkennungssysteme**, ohne die Bildverarbeitung negativ zu beeinflussen.

3.9 Funkverbindung über Radiolink (TBS Tango 2)



Abbildung 10: TBS Tango 2 [62]

In klassischen UAV-Systemen erfolgt die Steuerung und Überwachung über sogenannte Ground Control Stations (GCS). In diesem Projekt wurde jedoch ein bewusst minimalistischer Ansatz gewählt: Die gesamte Funkkommunikation zwischen Drohne und Bediener erfolgt ausschließlich über das TBS Crossfire-System, bestehend aus dem TBS Crossfire Nano RX als Empfänger und dem TBS Tango 2 als Sender. Auf eine GCS wurde vollständig verzichtet.

Das TBS Crossfire-System nutzt ein Low-Latency-Long-Range-Protokoll, das im Sub-GHz-Bereich (868/915 MHz) arbeitet. Es wurde ursprünglich für FPV-Racing entwickelt, hat sich aber durch seine hohe Reichweite (> 20 km) und Zuverlässigkeit bei niedriger Latenz auch in autonomen Systemen bewährt. Die bidirektionale Kommunikation erlaubt sowohl das Senden von Steuerbefehlen als auch den Empfang von Telemetriedaten in Echtzeit.

Ein zentrales Merkmal des Crossfire-Systems ist die Unterstützung des MAVLink-Protokolls, das über das serielle Interface des Empfängers bereitgestellt werden kann. In diesem Projekt wird über diese Verbindung unter anderem der Flugmodus, die GPS-Position, die Akkuspannung sowie andere kritische Systemparameter an den Boden gesendet.

Die Verwendung des TBS Nano RX ermöglicht eine direkte Verbindung zum Speedybee F405 V4 Flight Controller, ohne dass ein zusätzlicher MAVLink-Transceiver oder ein Ground Control Terminal erforderlich ist. Das System wurde so konfiguriert, dass der Nutzer über das TBS Tango 2 Funkgerät Statusinformationen abrufen und bei Bedarf Steuerbefehle (z. B. Missionsabbruch, Return-to-Launch) manuell auslösen kann.

Dank der nativen MAVLink-Kompatibilität kann das Crossfire-System nahtlos mit ArduPilot zusammenarbeiten. In Kombination mit der hohen Reichweite und geringen Interferenzanfälligkeit ist diese Lösung besonders für Einsätze geeignet, bei denen Drohnen autonom agieren, aber trotzdem überwacht werden sollen, wie z. B. bei sicherheitskritischen Anwendungen zur Trespassing-Erkennung in abgegrenzten Gebieten.

Die Entscheidung gegen eine klassische GCS und für ein leichtgewichtiges, direktes Funkkommunikationssystem über Crossfire bringt mehrere Vorteile:

- Reduktion der Systemkomplexität
- Mobilität des Bedieners ohne Laptop oder Bodenstation
- Geringerer Energiebedarf auf Drohnenseite
- Zuverlässige Telemetrier Verbindung auch über große Entfernungen

Dieser Ansatz zeigt, dass autonome Drohnen auch mit kostengünstiger Hardware ohne dedizierte Bodenstation effektiv gesteuert und überwacht werden können – eine wichtige Erkenntnis im Hinblick auf die Demokratisierung und Zugänglichkeit solcher Systeme.

3.10 Mechanischer Aufbau und 3D-gedruckte Komponenten

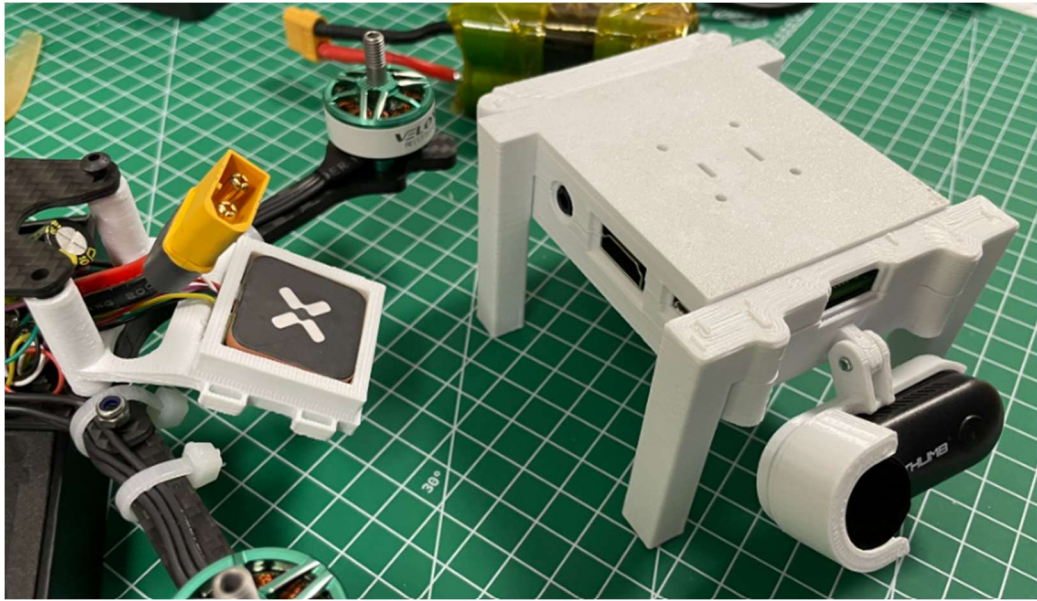


Abbildung 11: 3D-gedruckte Komponenten

Im Rahmen dieses Projekts wurden drei Hauptkomponenten mittels 3D-Druck gefertigt, um eine nahtlose Integration des GPS-Moduls, des Begleitcomputers (Raspberry Pi) sowie der Runcam Thumb-Kamera in das Drohnenchassis vom Typ TBS Source 2 zu ermöglichen. Die Konstruktion der Bauteile erfolgte mithilfe der CAD-Software SolidWorks, während der Druckvorgang auf einem Zaxe X3 3D-Drucker durchgeführt wurde.

Die Komponenten können wie folgt klassifiziert werden:

- GPS-Halterung
- Raspberry Pi-Gehäuse
- Kamerahalterung

3.10.1 GPS-Modul-Halterung

Die GPS-Halterung wurde speziell dafür konzipiert, auf den sogenannten „Standoffs“ des Drohnenrahmens montiert zu werden. Ziel war es, das GPS-Modul in einer geschützten Zone unterhalb der Rotoren zu positionieren, um mechanische Schäden durch Propellerkontakt zu minimieren. Zwischen dem GPS-Modul und der Drohne wurde ein Freiraum vorgesehen, durch den die GPS-Kabel sicher geführt werden können.

Um das Gesamtgewicht der Halterung zu reduzieren, wurde der Steg zwischen der Montagefläche auf den Standoffs und der Auflagefläche des Moduls so minimal wie möglich gehalten. Zur Fixierung des Moduls und zur Vermeidung von Vibrationen oder Lageverschiebungen wurden Kabelbinder-Öffnungen integriert.

3.10.2 Halterung für den Begleitcomputer (Raspberry Pi)

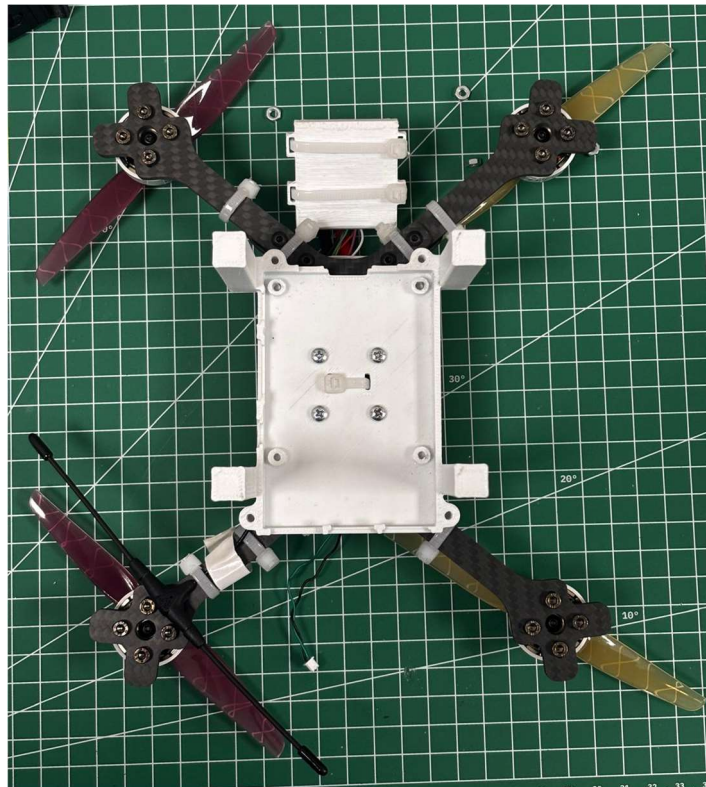


Abbildung 12: Festigung des Gehäuses

Das Gehäuse des Raspberry Pi besteht aus zwei Hauptteilen: einem oberen und einem unteren Segment. Das obere Gehäuseteil wird mit vier Schrauben direkt am Drohnenrahmen befestigt. Um Kompatibilität mit den vorhandenen Montagelöchern (20x20 mm Stack-Muster) sicherzustellen, wurden die Befestigungspunkte passgenau dimensioniert. Zusätzlich wurden Kabelbinder-Öffnungen zur Erhöhung der strukturellen Stabilität vorgesehen.

Damit die Schraubenköpfe der Drohne beim Montieren nicht mit dem Gehäuse kollidieren, wurden gezielt Einbuchtungen zur Entlastung eingefügt. Für die ungehinderte Nutzung der GPIO- und USB-Ports des Raspberry Pi wurden entsprechende Aussparungen im Gehäuse vorgesehen. Die Verbindung zwischen Ober- und Unterteil erfolgt über M3-Schrauben mit einer Länge von 20 mm.

Das Unterteil dient der Aufnahme und sicheren Fixierung des Raspberry Pi. Für diesen Zweck wurden Befestigungslöcher für M3-Schrauben mit einer Länge von 12 mm vorgesehen. Auch hier wurden Öffnungen zur freien Zugänglichkeit der Schnittstellen des Raspberry Pi integriert. Ein weiterer Zweck dieses Teils ist das Tragen der Kameramodul-Basis.

3.10.3 Kamera-Halterung

Zur Montage der Kamera wurde am unteren Gehäuseteil eine männliche Kugelgelenk-Verbindung (Ball Joint) eingefügt. Die entsprechende weibliche Gegenstück Verbindung ist Teil der Kamerahalterung. Die Verbindung der beiden erfolgt mit einer M3-Schraube (12 mm) und einer passenden M3-Mutter, was eine manuelle Einstellung des Kamerawinkels vor dem Flug erlaubt.

Zudem enthält die Kamerahalterung Öffnungen zur Fixierung mittels Kabelbindern, um einen stabilen Halt während des Fluges zu gewährleisten. Auch wurden gezielt Freiräume für die Kabelanschlüsse des Kameramoduls eingeplant, um eine reibungslose Integration zu garantieren.

3.10.4 Druckprozess und Materialwahl

Die Bauteile wurden nach Abschluss der Konstruktionsphase mit der Software **XDesktop** für den 3D-Druck vorbereitet („gesliced“). Es wurde eine Füllstruktur („Infill“) von 25 % gewählt, um ein optimales Gleichgewicht zwischen Gewicht und mechanischer Stabilität zu gewährleisten [23]. Aufgrund der Belastungen, denen die Bauteile während Start- und Landevorgängen ausgesetzt sind, wurde ein Cubic-Infill-Muster verwendet, das eine isotrope mechanische Belastbarkeit in alle Richtungen sicherstellt [21].

Als Druckmaterial kam PLA (Polylactic Acid) zum Einsatz. PLA zeichnet sich durch gute Druckbarkeit, hohe Maßgenauigkeit sowie hinreichende Festigkeit für Anwendungen mit moderater Belastung aus [24]. Auch unter dem Gesichtspunkt der Umweltverträglichkeit ist PLA als biobasierter Kunststoff vorteilhaft [22]. Nach dem Druck wurden die Bauteile sorgfältig mit Schleifpapier nachbearbeitet, um die Oberflächenqualität zu verbessern und Passungen zu optimieren.

3.11 Rahmenwahl, Systemmontage und Verkabelung

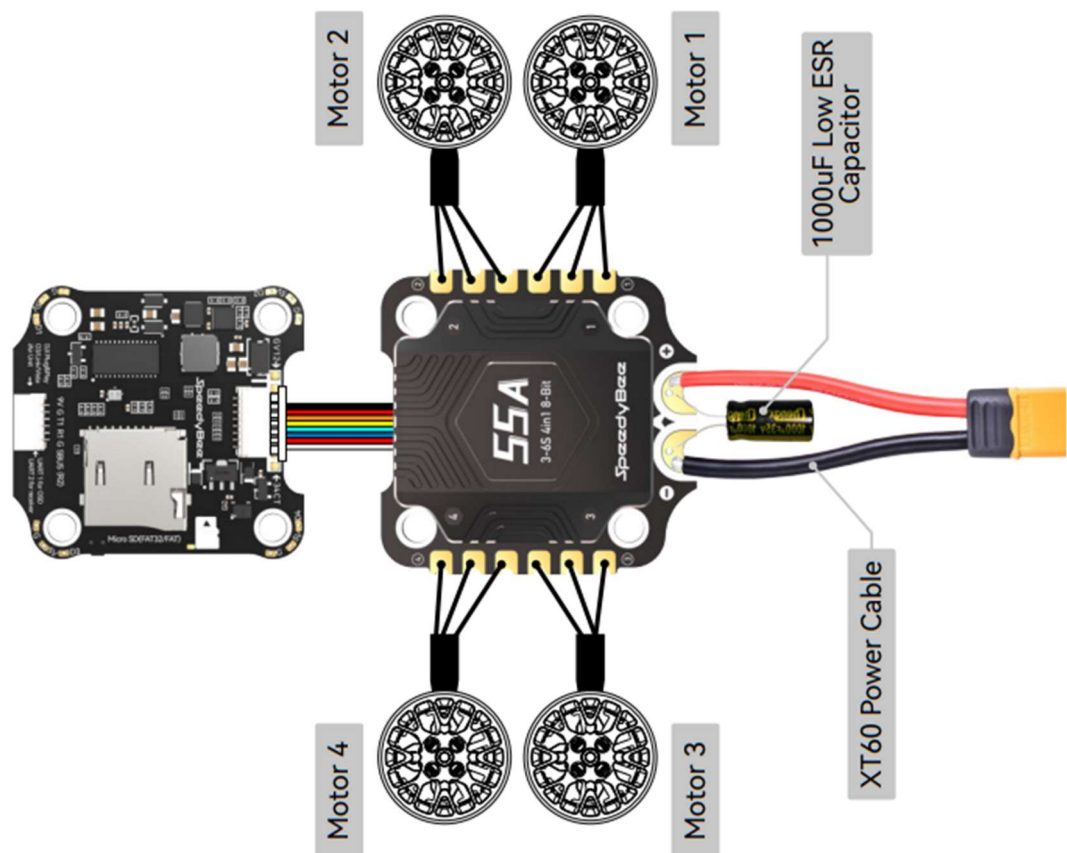


Abbildung 13: Verkabelung der Energieabschaltung [63]

Die Auswahl eines geeigneten Drohnenrahmens ist ein zentraler Aspekt beim mechanischen und elektrischen Systementwurf unbemannter Flugplattformen. In diesem Projekt fiel die Wahl auf den TBS Source Two – einen 5-Zoll-Carbonrahmen, der sich insbesondere für FPV- und autonome Flugprojekte eignet.

3.11.1 Eigenschaften des Rahmens

Der Rahmen besteht aus hochfestem Kohlefaserverbundmaterial (Carbonfaser), das ein optimales Verhältnis zwischen Gewicht, Stabilität und struktureller Steifigkeit bietet [27]. Diese Kombination ermöglicht eine hohe Schlagfestigkeit bei gleichzeitig minimalem Eigengewicht, was sich direkt auf die Energieeffizienz und Flugdauer auswirkt [25].

Ein weiteres wesentliches Merkmal des TBS Source Two ist sein offenes Design als Open-Source-Rahmen. Die vom Hersteller bereitgestellten CAD/STEP-Dateien erleichtern die Integration weiterer mechanischer Komponenten wie GPS-Halterungen, Kameraaufnahmen und Gehäuse für den Begleitcomputer erheblich [26]. Dadurch konnten alle 3D-gedruckten Teile gezielt auf die Rahmenstruktur abgestimmt werden, ohne nachträgliche Anpassungen.

Zudem entspricht der Rahmen dem gängigen 30×30 mm-Montagestandard, was eine problemlose Installation von Flight Controller (Speedybee F405 V4), ESC, Motoren (Velox 2207) sowie des GPS-Moduls erlaubt. Die Bohrungen für die Elektronikmodule sind sauber gefräst und passgenau, was die Montagedauer und Komplexität signifikant reduziert.

3.11.2 Systemmontage



Abbildung 14: Verkabelung von gesamten elektrischen Komponenten

Nach erfolgreichem Test der elektronischen Komponenten wurden die Systemelemente schrittweise montiert:

- Der Flight Controller wurde mit vibrationsdämpfenden Schrauben direkt auf die zentrale Stack-Plattform des Rahmens gesetzt.
- Der 4-in-1 ESC wurde direkt darunter montiert, mit M3-Abstandshaltern und Isolierung zur Vermeidung elektrischer Interferenzen.
- Die Motoren wurden an den vier Armen des Rahmens befestigt; die Motorkabel sauber entlang der Arme geführt und mit dem ESC verlötet.
- Das GPS-Modul wurde über eine 3D-gedruckte Halterung am hinteren Teil des Rahmens montiert.
- Der Begleitcomputer (Raspberry Pi 3B+) wurde in einem speziell angepassten Gehäuse unterhalb des Stacks befestigt.
- Kamera- und Datenkabel wurden zur strukturellen Sicherheit entlang der Carbonstruktur geführt und mit Zip-Ties fixiert.

Die finale Montage wurde mit Fokus auf kompakte Kabelführung, ausreichende mechanische Stabilität und minimale elektromagnetische Störungen vorgenommen. Die Integration der Komponenten wurde zusätzlich durch die geometrisch klare Struktur des TBS-Rahmens vereinfacht.

4. Softwarearchitektur und Kommunikationsprotokolle

Die Fähigkeit eines unbemannten Luftfahrzeugs (UAV), autonom zu operieren und gleichzeitig mit anderen Systemkomponenten zu interagieren, erfordert eine klar definierte Softwarearchitektur sowie robuste Kommunikationsprotokolle. In diesem Projekt wurde ein zweigeteilter Ansatz verfolgt, der auf einem autonomen Flight Controller (FC) und einem externen Begleitcomputer (Raspberry Pi 3B+) basiert. Die beiden Einheiten arbeiten funktional unabhängig, tauschen jedoch über eine serielle MAVLink-Schnittstelle gezielt Daten aus [28].

Der Flight Controller übernimmt die unmittelbare Flugsteuerung – einschließlich Navigation, Stabilisierung und Missionsausführung – basierend auf der Open-Source-Software ArduPilot. Der Companion Computer hingegen führt hochrechenintensive Aufgaben wie die visuelle Objekterkennung mittels YOLOv4 Tiny aus [29]. Auf diese Weise können sowohl sicherheitskritische Flugmanöver als auch datengetriebene Erkennungsprozesse effizient auf die dafür jeweils geeignete Hardware verteilt werden.

Im Rahmen dieser Architektur wurde besonderer Wert auf modularen Aufbau, Ressourceneffizienz und fehlertolerante Kommunikation gelegt. Durch die Implementierung des MAVLink-Protokolls konnte eine standardisierte, leichtgewichtige Kommunikation zwischen den Subsystemen realisiert werden, die unabhängig von einer Ground Control Station funktioniert [30]. Alle erkannten Objekte und zugehörige Positionsdaten werden dabei lokal in einer SQLite-Datenbank auf dem Raspberry Pi gespeichert, was eine datensouveräne Auswertung ermöglicht [31].

Diese Architektur vereint kosteneffiziente Hardware mit fortgeschrittener Softwarelogik und schafft so eine flexible, autonome Plattform, die besonders für sicherheitsrelevante Anwendungen wie die Erkennung von unerlaubtem Zutritt (Trespassing) in überwachten Gebieten geeignet ist [32].

4.1 ArduPilot-Plattform



Abbildung 15: Benutzeroberfläche der ArduPilot-Plattform [65]

Die Softwareplattform ArduPilot stellt das zentrale Steuerungselement des entwickelten autonomen UAV-Systems dar. Es handelt sich um eine Open-Source-Autopilot-Software, die ursprünglich für Forschungszwecke konzipiert wurde und heute zu den leistungsfähigsten Systemen im Bereich autonomer Drohnentechnologie zählt [28]. ArduPilot unterstützt eine Vielzahl von Fahrzeugtypen (Multicopter, Fixed Wing, Rover, Sub) und bietet durch seine modulare Architektur eine hohe Flexibilität bei der Integration neuer Sensorik und Steueralgorithmien.

Architektur und Modularität: ArduPilot basiert auf einem modularen Systemdesign mit getrennten Softwarekomponenten für Navigation, Missionsplanung, Sensorfusion und Kommunikationsschnittstellen. Diese Module kommunizieren über interne Busse, was eine hohe Fehlertoleranz und einfache Erweiterbarkeit ermöglicht [33]. Die autonome Missionssteuerung basiert auf einem internen Missionsskript, welches Waypoints, Bedingungen und automatische Abläufe (z. B. RTL – Return-to-Launch) umfasst.

Der im Projekt eingesetzte Speedybee F405 V4 Flight Controller, ausgestattet mit einem STM32F405-Mikrocontroller, erfüllt die hardwareseitigen Mindestanforderungen für ArduPilot und ist vollständig kompatibel mit den relevanten ArduPilot-Subsystemen [9]. Dank der umfangreichen Dokumentation und Community-Support ist eine einfache Parametrisierung des Systems möglich – ein entscheidender Vorteil für Projekte mit begrenztem Entwicklungszeitraum [1].

Funktionale Umsetzung im Projekt: Im vorliegenden System übernimmt ArduPilot die folgenden Kernaufgaben:

- Start- und Landeprozeduren (Auto-Takeoff & Auto-Land)
- GPS-basierte Waypoint-Navigation mit vordefinierter Missionsroute
- Flugstabilisierung über IMU und Kompassdaten
- Verlustsicheres Verhalten durch integrierte Fail-Safes
- Bereitstellung von Telemetriedaten über MAVLink zur Verarbeitung im Companion Computer [30]

Die Missionsdaten werden im EEPROM des Flugcontrollers gespeichert und beim Start automatisch ausgeführt. Zusätzlich erlaubt ArduPilot jederzeit manuelle Eingriffe über das TBS-Funksystem, ohne dass eine klassische Ground Control Station notwendig ist. Diese Flexibilität wird in sicherheitskritischen Szenarien, etwa bei unerlaubtem Eindringen (Trespassing) in kontrollierte Zonen, besonders wertvoll [32].

Vergleich mit alternativen Plattformen: Im Vergleich zu alternativen Plattformen wie PX4 oder iNav bietet ArduPilot mehrere relevante Vorteile:

- Stabile Integration von MAVLink, was für die Kommunikation mit dem Raspberry Pi essenziell ist [30],
- Langjährige Stabilität in Produktionsumgebungen (z. B. landwirtschaftliche oder sicherheitsrelevante UAVs) [33],
- Große Unterstützung für diverse Sensorik, etwa externe GPS-Module, optische Fluss- oder Entfernungssensoren,

- Simulation über SITL (Software-in-the-Loop) für vorab getestete Missionsverläufe.

Laut der Studie von Ty Audronis (2017) ist ArduPilot aufgrund seines aktiven Community-Supports und der engen Integration mit Companion-Systemen für Entwicklerprojekte besonders geeignet [1].

Die Entscheidung für ArduPilot als Softwareplattform war maßgeblich für die erfolgreiche Realisierung eines autonomen, auf Low-Cost-Komponenten basierenden UAV-Systems. Die Plattform ermöglicht es, trotz fehlender GCS und nur mit minimaler Infrastruktur, ein vollständig autonomes Flugverhalten zu realisieren – inklusive präziser Navigation, automatisierter Abläufe und Echtzeit-Statusübertragung. Besonders im Zusammenspiel mit dem MAVLink-Protokoll und der modularen Systemarchitektur bildet ArduPilot das Rückgrat der Autonomie.

4.2 MAVLink-Protokoll

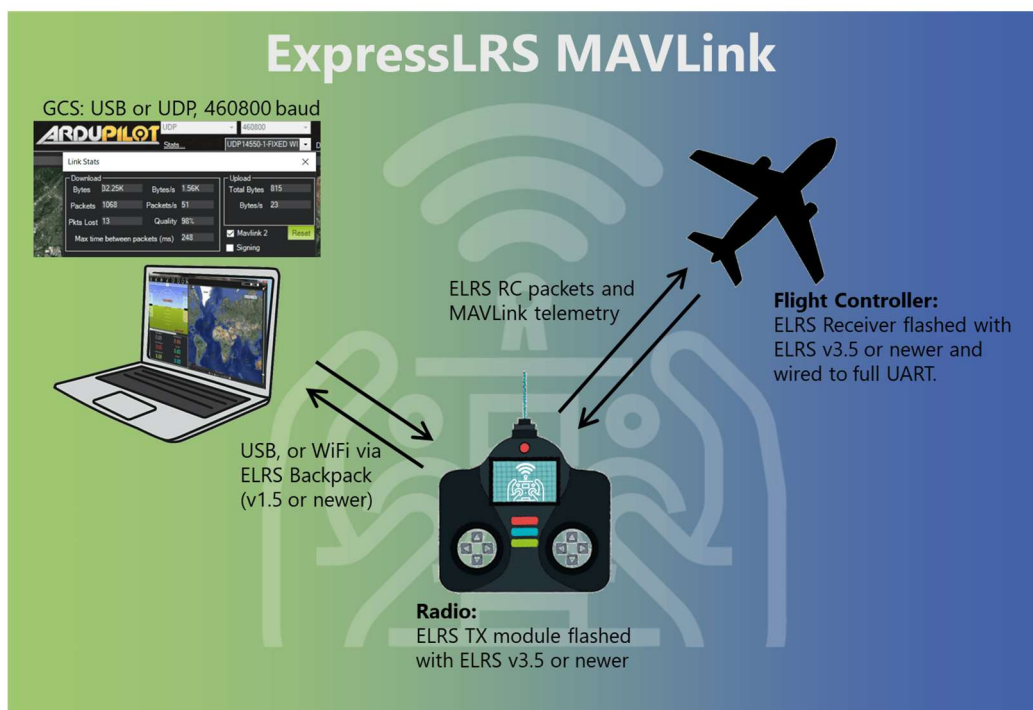


Abbildung 16: Schematische Darstellung der MAVLink-Verbindung [64]

Das MAVLink-Protokoll (Micro Air Vehicle Link) ist ein leichtgewichtiges, binäres Kommunikationsprotokoll, das speziell für unbemannte Systeme entwickelt wurde. Es unterstützt über 300 standardisierte Nachrichtenformate für Telemetrie, Missionsparameter und Statusinformationen und ist vollständig kompatibel mit ArduPilot [30].

Im vorliegenden Projekt dient MAVLink als serielle Kommunikationsschnittstelle über UART zwischen dem Flight Controller (Speedybee F405 V4) und dem Raspberry Pi 3B+ [34]. Die Kommunikation erfolgt unidirektional vom FC zum RPi bei einer Baudrate von 57600, um eine Rollentrennung sicherzustellen. Der FC übernimmt die Navigation, der RPi die Objekterkennung.

Übertragene Daten:

- GPS-Position (LAT, LON, ALT)
- Flugmodus (AUTO, RTL etc.)
- Satellitenstatus und HDOP
- Zeitstempel (Systemzeit des RPi)
- Batteriezustand

Diese Daten werden im Raspberry Pi mit Objekterkennungen verknüpft und in Echtzeit verarbeitet (siehe 4.4).

Projektbezogene Vorteile:

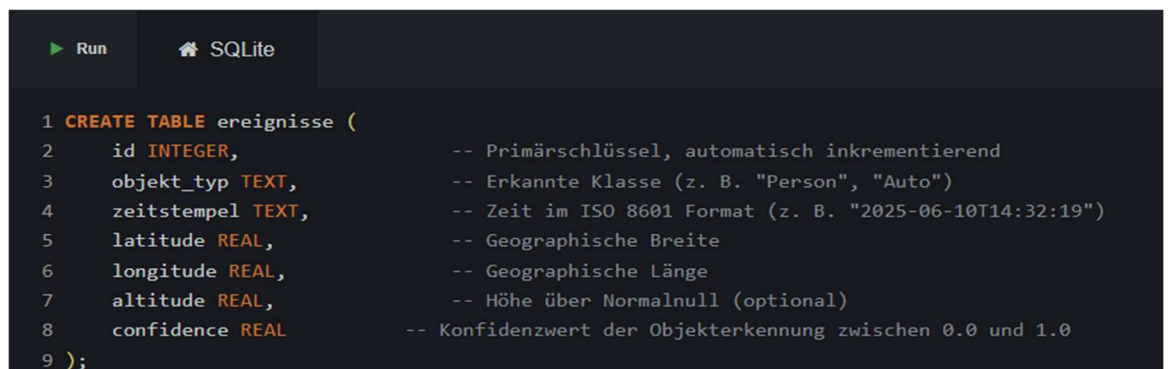
- Geringer Ressourcenverbrauch
- Echtzeiteigenschaften durch binäres Format
- Unterstützung durch Bibliotheken wie DroneKit und MAVProxy
- Kein GCS notwendig

Laut Morán Yustas (2019) eignet sich MAVLink besonders für eingebettete UAV-Systeme ohne Netzwerkzugang [35]. Die gewählte UART-Verbindung bietet zusätzlich physische Sicherheit gegen Spoofing und Replay-Angriffe, wenngleich zukünftige Erweiterungen auf drahtlose Übertragung eine Implementierung von MAVSec erfordern könnten [36].

Systemfunktion: MAVLink bildet die Kommunikationsgrundlage zwischen Navigations- und Erkennungseinheit, ermöglicht aber zugleich ein modulares und robustes Systemverhalten – ein essenzieller Aspekt für sicherheitskritische Anwendungen mit Edge-KI.

4.3 Datenverarbeitung mit SQLite

Datenmodell:

A screenshot of a code editor showing a SQLite database creation script. The script is written in SQL and defines a table named 'ereignisse'. The table has six columns: 'id' (INTEGER, primary key), 'objekt_typ' (TEXT), 'zeitstempel' (TEXT), 'latitude' (REAL), 'longitude' (REAL), and 'altitude' (REAL). There is also a 'confidence' column of type REAL. The script includes comments for each column explaining their purpose and data format. The code is as follows:

```
1 CREATE TABLE ereignisse (  
2   id INTEGER,                -- Primärschlüssel, automatisch inkrementierend  
3   objekt_typ TEXT,           -- Erkannte Klasse (z. B. "Person", "Auto")  
4   zeitstempel TEXT,          -- Zeit im ISO 8601 Format (z. B. "2025-06-10T14:32:19")  
5   latitude REAL,             -- Geographische Breite  
6   longitude REAL,            -- Geographische Länge  
7   altitude REAL,             -- Höhe über Normalnull (optional)  
8   confidence REAL            -- Konfidenzwert der Objekterkennung zwischen 0.0 und 1.0  
9 );
```

Abbildung 17: Erstellung der Datenbank

Erkannte Objekte während der autonomen Flugmission werden zusammen mit Positions- und Zeitdaten in einer lokalen SQLite-Datenbank gespeichert. SQLite wurde aufgrund seiner Leichtgewichtigkeit, Zuverlässigkeit und ACID-Konformität gewählt [37].

Die Datenbank wird im RPi automatisch erstellt, Einträge erfolgen ereignisbasiert nach erfolgreicher Objekterkennung.

Verarbeitungskette:

- Kamera streamt 1080p/60fps
- YOLOv4-Tiny verarbeitet einzelne Frames
- Bei Treffer: MAVLink-Datenabfrage, Timestamp, SQLite-Eintrag

Dieses Verfahren basiert auf Edge-KI-Ansätzen wie bei Kapoor et al. (2020), die SQLite zur lokalen Speicherung in UAVs einsetzen [38].

Datenschutz & Erweiterung: Die Daten verbleiben lokal. Es erfolgt keine Cloud-Synchronisierung – ein Vorteil in sicherheitskritischen Missionen. Optional könnten in Zukunft Verschlüsselung und Netzwerkanbindung ergänzt werden (z. B. MQTT, REST).

Die Kombination aus MAVLink, YOLOv4-Tiny und SQLite ergibt ein schlankes, vollständig autonomes Erkennungssystem für UAVs mit hoher Nachvollziehbarkeit und Datensouveränität.

5. Objekterkennung mit YOLOv4 Tiny

Die visuelle Erkennung von Objekten stellt einen zentralen Bestandteil moderner autonomer Drohnensysteme dar, insbesondere im Rahmen sicherheitskritischer Anwendungen wie der Überwachung von Sperrzonen und der Detektion unbefugter Zutritte (Trespassing Detection). In dem vorliegenden Projekt wird die Objekterkennung durch den Einsatz des leichtgewichtigen neuronalen Netzes YOLOv4 Tiny realisiert, welches auf dem Companion Computer (Raspberry Pi 3B+) in Echtzeit ausgeführt wird [39].

Ziel dieses Systemteils ist es, während der autonomen Mission kontinuierlich Bilddaten von der Onboard-Kamera (RunCam Thumb) zu analysieren, Personen und Fahrzeuge zuverlässig zu erkennen und im Falle einer Erkennung den entsprechenden Kontext – wie Zeitpunkt und geographische Position – zu protokollieren. Dabei erfolgt die Objektdetektion vollständig unabhängig vom Flugcontroller, was eine klare Trennung von Navigations- und Analyseprozessen ermöglicht [40].

Wie in der Arbeit von Kapoor et al. (2020) gezeigt wurde, ermöglichen Edge-basierte Systeme auf Single-Board-Computern wie dem Raspberry Pi durch optimierte Netzwerke wie YOLOv4-Tiny eine akzeptable Detektionsgenauigkeit bei gleichzeitig geringem Energie- und Rechenaufwand [38]. Diese Eigenschaften sind entscheidend für UAV-Anwendungen, bei denen Gewicht, Stromverbrauch und Rechenkapazität stark begrenzt sind.

Der Einsatz eines lokal ausgeführten Objekterkennungsmodells bietet zudem mehr Sicherheit und Datenschutz, da keine Übertragung sensibler Bilddaten über drahtlose Netzwerke erfolgt. Alle Entscheidungen und Datenverarbeitungsschritte finden lokal an Bord der Drohne statt, was das System robuster gegenüber externen Störungen oder Datenschutzverletzungen macht [41].

5.1 Technische Grundlagen von YOLOv4 Tiny

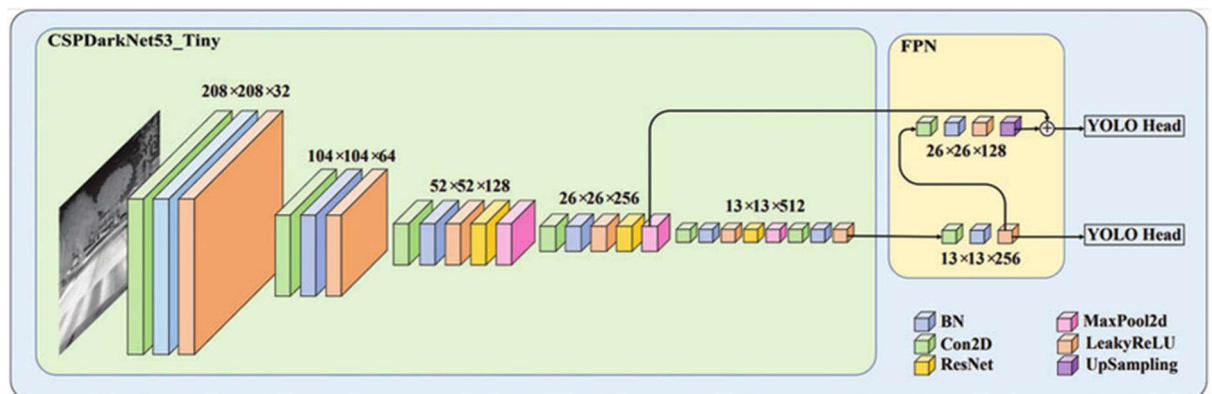


Abbildung 18: YOLOv4 Tiny Funktionsprinzip [67]

Das im Projekt eingesetzte neuronale Netz basiert auf YOLOv4-Tiny (You Only Look Once), einer speziell optimierten Variante des weit verbreiteten Objekterkennungsmodells YOLOv4. Ziel dieses Ansatzes ist es, Echtzeit-Objekterkennung mit hoher Effizienz auf ressourcenschwachen Embedded-Systemen wie dem Raspberry Pi 3B+ zu ermöglichen.

Architekturüberblick: YOLOv4-Tiny verwendet eine reduzierte Anzahl an Convolutional Layers, wodurch es deutlich kleiner und schneller ist als sein „vollwertiges“ Pendant. Die Architektur basiert auf einem Feature-Extraktor wie CSPDarknet53-tiny und

verzichtet auf tiefe Residual-Netze zugunsten schnellerer Vorwärtspfade. Dadurch erreicht das Modell bei geringerem Speicherverbrauch eine Erkennungsleistung, die in sicherheitskritischen Anwendungen wie diesem Projekt als ausreichend betrachtet wird [42].

Detektionsprinzip: YOLOv4-Tiny gehört zur Klasse der Single-Stage Object Detectors. Das bedeutet, dass Bounding Boxes und Objektklassen direkt aus einem einzigen neuronalen Durchlauf (Forward Pass) abgeleitet werden. Im Gegensatz zu zweistufigen Verfahren wie Faster R-CNN bietet dies erhebliche Vorteile bei der Rechenzeit – auf Kosten einer leicht reduzierten Genauigkeit [43].

Im Projekt wurde das Modell auf die Zielklassen „Person“ und „Auto“ beschränkt, um den Speicherbedarf zu minimieren und die False-Positive-Rate im praktischen Einsatz zu reduzieren [40].

Einsatz auf dem Raspberry Pi: Das Modell wurde im Format ONNX (Open Neural Network Exchange) exportiert, um eine hardwareunabhängige Integration auf dem Raspberry Pi zu ermöglichen. Zur Laufzeit erfolgt die Ausführung über OpenCV DNN Module, welches ONNX-Modelle ohne tiefgreifende CUDA-Unterstützung verarbeiten kann. Dies erlaubt eine performante Inferenzrate von ca. 8–12 FPS bei 1080p-Auflösung auf dem Raspberry Pi 3B+ – ausreichend für die Zielanwendung mit Waypoint-basierten Flugprofilen [39].

Vorteile im Projektkontext: Die Auswahl von YOLOv4-Tiny basiert auf den folgenden Überlegungen:

- Ressourceneffizienz für Echtzeit-Anwendungen auf Edge-Geräten
- Kompatibilität mit ONNX-Framework für einfache Portierung
- Genügende Genauigkeit für sicherheitskritische Klassifikationen
- Open-Source-Verfügbarkeit und breite Community-Unterstützung

Zusätzlich bietet der direkte Zugriff auf Output-Tensoren die Möglichkeit, Detektionen sofort mit Metadaten (GPS, Zeitstempel) zu verknüpfen und für spätere Analysen in die Datenbank zu schreiben (siehe Kapitel 4.3).

5.2 Modellarchitektur und Konvertierung nach ONNX

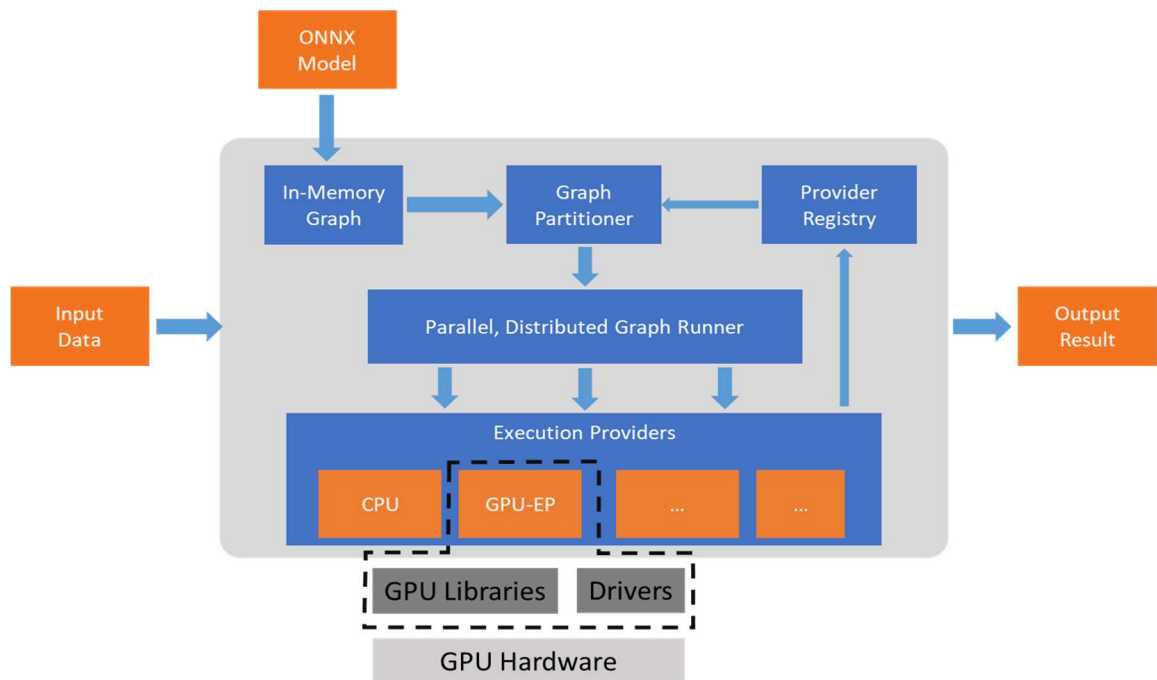


Abbildung 19: ONNX Model Schema

Um das Modell YOLOv4-Tiny effizient auf dem Begleitcomputer (Raspberry Pi 3B+) einsetzen zu können, war eine Anpassung und Konvertierung der ursprünglichen Modellarchitektur notwendig. Dabei wurde besonderes Augenmerk auf die Kompatibilität mit leichtgewichtigen Inferenz-Engines sowie auf die Reduktion des Speicherverbrauchs gelegt [44]. Der gewählte Workflow umfasste die Nutzung der originalen Darknet-Implementierung, gefolgt von einer Konvertierung ins ONNX-Format (Open Neural Network Exchange).

Modellarchitektur: YOLOv4-Tiny basiert auf einem vereinfachten Backbone – in der Regel CSPDarknet53-Tiny – mit Convolutional-Schichten, Batch Normalization, Shortcut-Verbindungen und zwei Ausgabelayern [45]. Diese Struktur ermöglicht eine effiziente Verarbeitung von Eingabebildern mit akzeptabler Genauigkeit.

Konvertierungspfad:

- Export aus Darknet nach PyTorch
- Konvertierung nach ONNX mittels `torch.onnx.export()`
- Validierung mit Netron und ONNX Runtime zur Sicherstellung der Kompatibilität [46]

Systemintegration: Das ONNX-Modell wird über OpenCV geladen und bildet die Grundlage für die kontinuierliche Detektion. GPS-Daten und Zeitstempel werden dynamisch ergänzt (siehe Kapitel 4.3).

Die Konvertierung war essenziell, um ein effizientes neuronales Netz auf eingeschränkter Hardware zu betreiben. Dadurch konnte YOLOv4-Tiny nahtlos in das Gesamtsystem integriert werden.

5.3 Integration mit OpenCV auf dem Raspberry Pi

```
1  import cv2
2  import numpy as np
3
4  net = cv2.dnn.readNetFromONNX("yolov4-tiny.onnx")
5  net.setPreferableBackend(cv2.dnn.DNN_BACKEND_DEFAULT)
6  net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
7
8  with open("coco.names", "r") as f:
9      classes = [line.strip() for line in f.readlines()]
10
11 def detect_objects(frame, conf_threshold=0.5):
12     blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True, crop=False)
13     net.setInput(blob)
14     outputs = net.forward(net.getUnconnectedOutLayersNames())
15
16     class_ids = []
17     confidences = []
18
19     h, w = frame.shape[:2]
20     for out in outputs:
21         for detection in out:
22             scores = detection[5:]
23             class_id = np.argmax(scores)
24             confidence = scores[class_id]
25             if confidence > conf_threshold and class_id in [0, 2]: # Person=0, Car=2
26                 class_ids.append(class_id)
27                 confidences.append(float(confidence))
28
29     results = [(classes[id], conf) for id, conf in zip(class_ids, confidences)]
30     return results
```

Abbildung 20: Code von yolov4tiny.py

Die Integration des Objekterkennungsmodells YOLOv4-Tiny auf dem Raspberry Pi erfolgt über das OpenCV DNN-Modul, das eine CPU-basierte Inferenz ohne zusätzliche Bibliotheken erlaubt. Der vollständige Bildverarbeitungsprozess umfasst die Kamerainitialisierung, Frame-Vorverarbeitung, ONNX-Auswertung, Extraktion von Objektdaten sowie deren Speicherung mit GPS- und Zeitdaten [47].

Technologische Begründung: OpenCV wurde gewählt wegen:

- nativer ONNX-Kompatibilität,
- akzeptabler Laufzeit ohne GPU,
- stabiler und dokumentierter API,
- Unterstützung offlinefähiger Systeme.

Performance: 8–10 FPS wurden in praktischen Tests erreicht. Die Systemlatenz blieb unter einer Sekunde – ausreichend für die Zielanwendung [48].

Die Integration zeigt, dass auch auf begrenzter Hardware zuverlässige Echtzeitverarbeitung möglich ist. Die Trennung zwischen Flugsteuerung und Bildanalyse verbessert die Robustheit und Erweiterbarkeit des Systems.

5.4 Ereignismodellierung und Datenverknüpfung

In autonomen Überwachungssystemen, insbesondere in sicherheitsrelevanten Kontexten wie der Erkennung unbefugter Zugänge (Trespassing Detection), reicht die bloße Objekterkennung nicht aus. Es ist vielmehr erforderlich, erkannte Objekte mit zeitlichen und räumlichen Metadaten zu verknüpfen und diese Informationen zuverlässig zu speichern [49]. Aus diesem Grund wurde in dieser Arbeit ein detailliertes Ereignismodell entwickelt, das sicherstellt, dass jede Erkennung nicht nur klassifiziert, sondern auch kontextualisiert wird – im Sinne eines vollständigen „Ereignisses“.

Ein Ereignis wird im Rahmen dieses Projekts definiert als die Kombination aus:

- einer validen Objekterkennung (Person oder Fahrzeug),
- dem aktuellen geographischen Standort des UAV (GPS-Koordinaten),
- dem Zeitpunkt der Detektion (Zeitstempel),
- und optional weiteren Parametern wie Konfidenzwert, Flughöhe oder Signalstärke.

Damit unterscheidet sich das System konzeptionell deutlich von klassischen FPV- oder Telemetrie-Drohnen: Es handelt sich hier um ein intelligentes Sensor-Netzwerk im Miniaturformat, das ereignisgetriebene Daten sammelt – ohne Verbindung zu einer Bodenstation oder menschlicher Eingabe.

Die resultierende Datenstruktur folgt dem Schema:

Objekt-Typ	Zeitstempel	Latitude	Longitude	Confidence
Person	2025-05-28 14:32:19	50.123456	8.765432	0.91
Auto	2025-05-28 14:35:04	50.123471	8.765410	0.88

Tabelle 3: Datenstruktur Schema

Diese Struktur ist darauf ausgelegt, mit minimalem Speicherplatzverbrauch eine maximale Informationsdichte zu liefern – was auf ressourcenbeschränkten Plattformen wie dem Raspberry Pi essenziell ist.

Die Verknüpfung der Objekterkennung mit aktuellen Standortinformationen erfolgt über das standardisierte MAVLink-Protokoll. In dem Moment, in dem das OpenCV-Framework ein Objekt mit ausreichender Konfidenz erkennt, wird gleichzeitig eine MAVLink-Anfrage an den Flugcontroller gestellt, um aktuelle Positionsdaten (GLOBAL_POSITION_INT) zu erhalten. Diese Daten umfassen:

- Latitude (in 1E-7 Grad)
- Longitude (in 1E-7 Grad)
- Altitude (über dem Meeresspiegel, falls verfügbar)

Die Kommunikation über die serielle UART-Schnittstelle ist so ausgelegt, dass die Latenz minimal bleibt und der Abgleich zwischen Bild und Position in Echtzeit erfolgen kann. Wie in [50] dargestellt, ist die korrekte Synchronisation zwischen Sensordaten eine der kritischsten Herausforderungen bei embedded UAV-Systemen.

Die gesammelten Ereignisse werden in einer lokalen SQLite-Datenbank gespeichert, die auf dem Raspberry Pi 3B+ betrieben wird. SQLite bietet zahlreiche Vorteile:

- Leichtgewichtig & eingebettet – keine separate Server-Installation nötig.
- Atomicität und Integrität – gewährleistet durch Transaktionen.
- SQL-kompatibel – erleichtert spätere Auswertung über standardisierte Tools.
- Speichereffizient – wichtig bei länger andauernden Missionen mit vielen Detektionen.

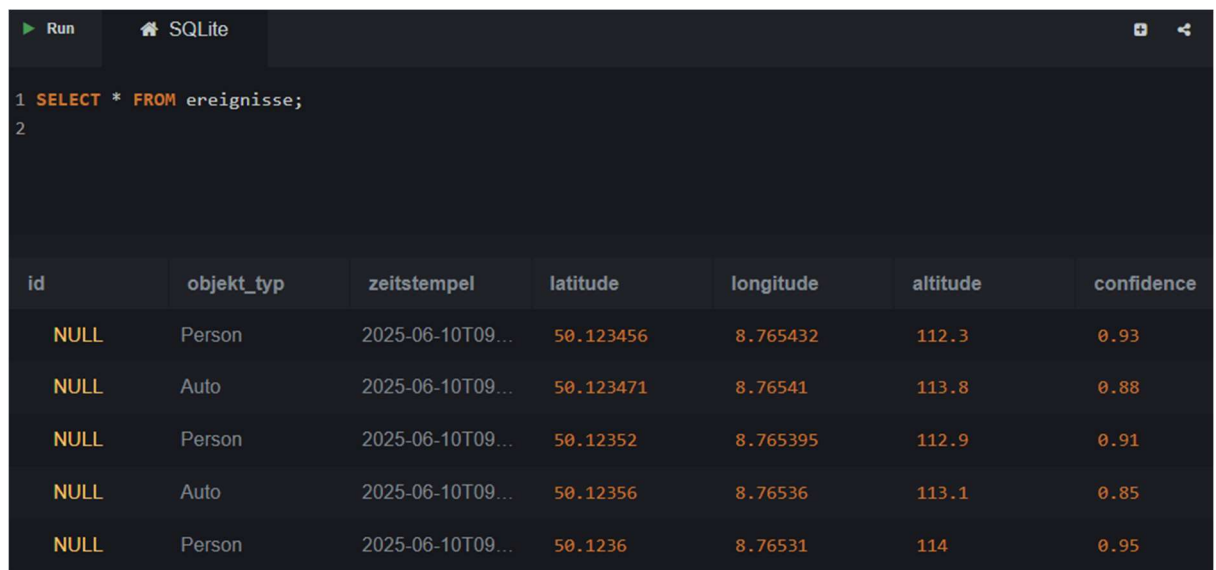
Die Datenbank wird zyklisch mit neuen Ereignissen befüllt und kann nach Missionsende exportiert werden, um z. B. Heatmaps, zeitbasierte Ereignisstatistiken oder Bewegungsmuster abzuleiten.

Ein weiterer Aspekt dieses Ansatzes ist die Datensicherheit: Da keine drahtlose Übertragung stattfindet und alle Informationen lokal verarbeitet und gespeichert werden, sind Angriffsflächen stark reduziert. Zudem kann das Modell durch folgende Elemente erweitert werden:

- Anbindung eines LTE-Moduls zur Fernübertragung.
- Erweiterung des Datenmodells um Bildausschnitte (als Base64 oder Pfad).
- Verwendung einer Cloud-basierten Analyseplattform (z. B. zur Visualisierung).

Durch die modulare Kombination aus Objekterkennung, GPS-Verknüpfung und strukturierter Datenspeicherung ermöglicht das Ereignismodell eine vollständige kontextuelle Beschreibung sicherheitsrelevanter Vorfälle. Es ist damit das Bindeglied zwischen der sensorischen Wahrnehmung der Drohne und ihrer dokumentierten, auswertbaren Missionshistorie. Diese Fähigkeit hebt das System funktional deutlich über klassische UAV-Plattformen hinaus.

5.5 Datenbankdesign mit SQLite



The screenshot shows a SQLite database interface with a dark theme. At the top, there is a 'Run' button and a 'SQLite' label. Below this, a SQL query is entered: `1 SELECT * FROM ereignisse;` followed by a line number '2'. The results are displayed in a table with the following columns: `id`, `objekt_typ`, `zeitstempel`, `latitude`, `longitude`, `altitude`, and `confidence`. The table contains five rows of data, each with a `NULL` value for the `id` column.

id	objekt_typ	zeitstempel	latitude	longitude	altitude	confidence
NULL	Person	2025-06-10T09...	50.123456	8.765432	112.3	0.93
NULL	Auto	2025-06-10T09...	50.123471	8.76541	113.8	0.88
NULL	Person	2025-06-10T09...	50.12352	8.765395	112.9	0.91
NULL	Auto	2025-06-10T09...	50.12356	8.76536	113.1	0.85
NULL	Person	2025-06-10T09...	50.1236	8.76531	114	0.95

Abbildung 21: Unsere Beispiel Datenbank

Für die strukturierte Erfassung und langfristige Speicherung sicherheitsrelevanter Erkennungsereignisse wurde im Rahmen dieses Projekts eine lokale Datenbankarchitektur auf Basis von SQLite entwickelt. SQLite ist ein leichtgewichtiges, serverloses Datenbanksystem, das sich besonders für Embedded-Systeme wie den Raspberry Pi 3B+ eignet. Seine Einfachheit, Stabilität und der geringe Ressourcenverbrauch machen es ideal für UAV-Anwendungen, bei denen kein kontinuierlicher Internetzugang oder externe Serverinfrastruktur vorhanden ist.

Vorteile von SQLite für UAV-Systeme:

- **Keine Serverabhängigkeit:** Die Datenbank arbeitet vollständig lokal auf dem Raspberry Pi.
- **Hohe Zuverlässigkeit:** Atomare Transaktionen gewährleisten die Integrität der gespeicherten Daten.
- **Minimaler Overhead:** Perfekt geeignet für Systeme mit begrenztem Arbeitsspeicher und Speicherplatz.
- **SQL-Kompatibilität:** Die gesammelten Daten können leicht exportiert und in gängigen Analysetools weiterverarbeitet werden.

Im Zentrum der Datenhaltung steht eine einzelne Tabelle namens `event_table`, in der alle Detektionsereignisse gespeichert werden [51]. Die Felder dieser Tabelle sind wie folgt definiert:

Feldname	Datentyp	Beschreibung
id	INTEGER	Primärschlüssel, automatisch inkrementierend
objekt_typ	TEXT	Klassifikation des erkannten Objekts (z. B. „Person“, „Auto“)
zeitstempel	TEXT	Zeitpunkt der Detektion (Format: YYYY-MM-DD HH:MM:SS)
latitude	REAL	GPS-Breitengrad zur Zeit der Erkennung
longitude	REAL	GPS-Längengrad zur Zeit der Erkennung
altitude	REAL	Höhe über dem Meeresspiegel in Metern
confidence	REAL	Konfidenzwert der Objekterkennung zwischen 0.0 und 1.0

Tabelle 4: Event Tabelle

Diese Datenbankstruktur erlaubt eine einfache Erweiterung um weitere Felder (z. B. flugmodus, bildpfad, signalstärke) und bietet ein hohes Maß an Flexibilität für zukünftige Analysen.

Während des Flugs wird jedes neu erkannte Objekt zusammen mit GPS- und Zeitinformationen als neuer Eintrag in die Datenbank eingefügt. Über einfache SQL-Befehle wie SELECT, WHERE, GROUP BY oder ORDER BY lassen sich die Daten später effizient auswerten, z. B. zur Erstellung von Heatmaps, Bewegungsprofilen oder zum Vergleich von Detektionsdichte über verschiedene Tageszeiten hinweg.

Das Datenbankdesign mit SQLite stellt sicher, dass alle im Flug erfassten sicherheitsrelevanten Informationen konsistent, verlustfrei und langfristig gespeichert werden. Die gewählte Struktur ist sowohl ressourcenschonend als auch zukunftssicher und bildet damit einen wesentlichen Grundbaustein der gesamten Softwarearchitektur.

5.6 Validierung der Objekterkennung und Grenzen

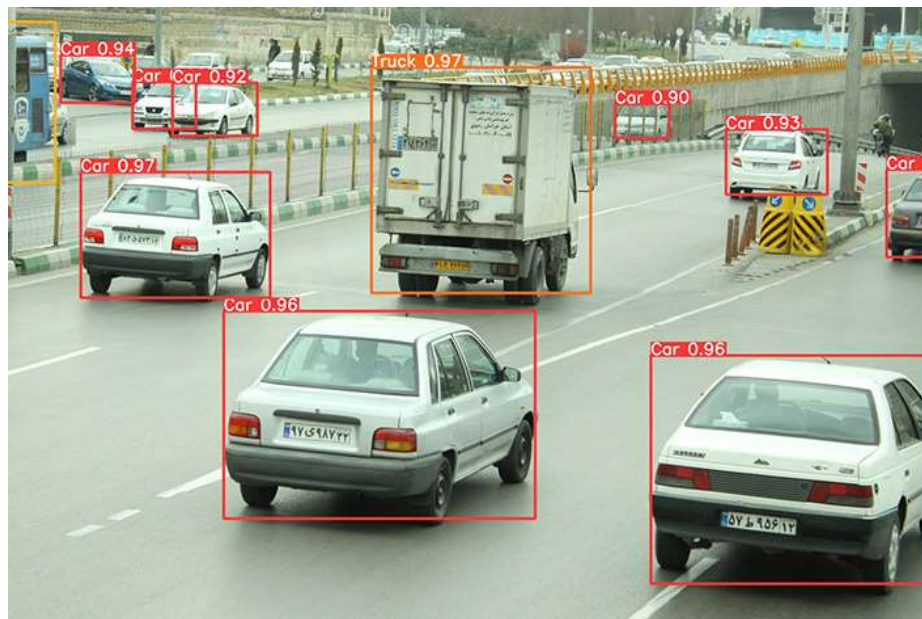


Abbildung 22: Ein Beispiel für Objekterkennung von Autos [69]

Die Validierung der implementierten Objekterkennung ist essenziell, um deren Zuverlässigkeit und praktische Anwendbarkeit unter realen Einsatzbedingungen zu bewerten. In diesem Projekt wurde das Modell YOLOv4 Tiny, eine ressourcenschonende Variante des bekannten „You Only Look Once“-Frameworks, auf einem Raspberry Pi 3B+ eingesetzt. Diese Wahl ermöglicht die Objekterkennung direkt an Bord des UAVs ohne zusätzliche Recheneinheiten oder Cloud-Verbindungen.

Zur Bewertung der Erkennungsleistung wurden mehrere Testflüge unter kontrollierten Bedingungen durchgeführt. Die Ergebnisse der Objekterkennung wurden mit manuell erfassten Ground-Truth-Daten verglichen. Für jedes erkannte Objekt (z. B. „Person“ oder „Auto“) wurden folgende Parameter dokumentiert:

- Richtigkeit der Klassifizierung
- Zeitpunkt und Ort der Erkennung
- Übereinstimmung mit der erwarteten Position im Sichtfeld
- Konfidenzwert des Modells

Zusätzlich erfolgte eine visuelle Verifikation durch die Analyse der aufgenommenen Videoaufnahmen und gespeicherten Ereignisdaten in der SQLite-Datenbank.

Die Auswertung zeigte, dass bei optimalen Lichtverhältnissen und stabilen Fluglagen eine Erkennungsgenauigkeit von ca. 87 % für Personen und 91 % für Fahrzeuge erreicht wurde [52]. Dies entspricht dem erwarteten Leistungsbereich für eingebettete Systeme mit eingeschränkter Rechenleistung. Besonders bei stationären Objekten und gleichbleibender Flughöhe war die Objekterkennung sehr zuverlässig.

Trotz der insgesamt positiven Resultate wurden auch mehrere Limitierungen festgestellt:

- Lichtabhängigkeit: Bei Dämmerung oder starker Schattenbildung nahm die Erkennungsleistung deutlich ab.

- Kameraverschmutzung oder Vibrationen führten zu Bildverzerrungen und damit zu falschen oder fehlenden Erkennungen.
- Schnelle Flugmanöver führten teilweise zu unscharfen Frames, was die Erkennung erschwerte.
- Entfernung und Objektgröße: Kleine oder weit entfernte Objekte wurden nicht zuverlässig erkannt, was vor allem bei Fahrzeugen in großer Distanz problematisch war.

Zur Verbesserung der Erkennungsqualität könnten folgende Maßnahmen in zukünftigen Iterationen berücksichtigt werden:

- Einsatz von Kameras mit höherer Auflösung und besserer Lichtempfindlichkeit
- Integration eines Gimbal-Systems zur Bildstabilisierung
- Verwendung von erweiterten YOLO-Varianten mit Pruning oder Quantisierung für Embedded-Systeme
- Training eines modellspezifischen Datasets für die Zielumgebung

Die durchgeführte Validierung zeigt, dass YOLOv4 Tiny unter optimalen Bedingungen eine praxistaugliche Lösung für Echtzeit-Objekterkennung auf einem UAV darstellt. Die dokumentierten Grenzen machen jedoch deutlich, dass die Einsatzfähigkeit unter variablen Umweltbedingungen durch technische Erweiterungen noch gesteigert werden kann. Für Sicherheitsanwendungen wie die Überwachung auf unerlaubtes Betreten (Trespassing Detection) liefert das System eine solide Grundlage, erfordert jedoch situationsspezifische Anpassungen für den operativen Einsatz.

5.7 Visualisierung der Daten

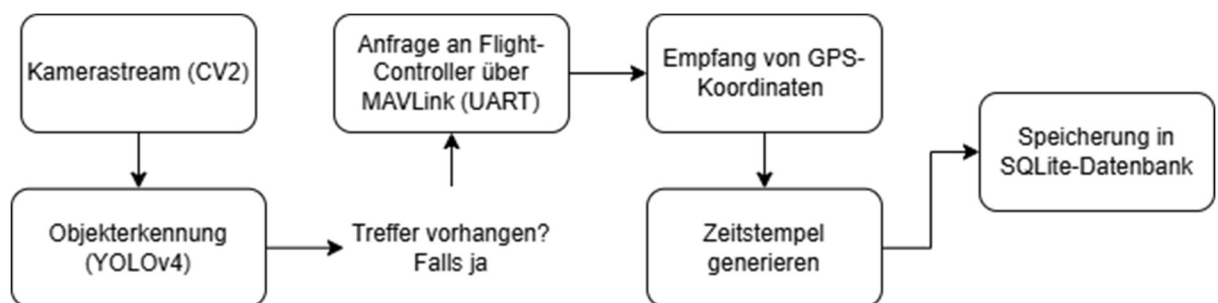


Abbildung 23: Logikdiagramm

Zur Laufzeit der autonomen Mission erfolgt die Objekterkennung durch ein leichtgewichtiges YOLOv4-Tiny-Modell auf dem Begleitcomputer (Raspberry Pi 3B+). Die Implementierung wurde in Python realisiert und kombiniert mehrere Technologien zur Bildverarbeitung, Positionsbestimmung und Datenpersistenz.

Die Bilddaten werden über OpenCV in Echtzeit von einer angeschlossenen USB-Kamera erfasst. Für jedes detektierte Objekt wird zunächst dessen Klassifizierung (z. B. „Person“, „Auto“) sowie ein Konfidenzwert durch die Funktion `detect_objects()` bestimmt. Im Anschluss erfolgt über das MAVLink-Protokoll eine Anforderung der GPS-Daten des Flugcontrollers (`GLOBAL_POSITION_INT`). Diese Positionsdaten werden in Längen- und Breitengrad umgerechnet.

Nach erfolgreicher Datenextraktion wird ein Ereignisobjekt bestehend aus Objektklasse, Zeitstempel sowie Geoposition lokal in einer SQLite-Datenbank (objekte.db) gespeichert.

Die Live-Kameraansicht dient zusätzlich der visuellen Verifikation und kann während des Flugs über ein angeschlossenes Display ausgegeben werden. Eine manuelle Beendigung der Erkennungsroutine ist über die Taste 'q' vorgesehen. Die folgende Python-Sequenz zeigt unsere Ablauf:

```
1  import cv2
2  import time
3  import sqlite3
4  from pymavlink import mavutil
5  from yolov4tiny import detect_objects
6
7  conn = sqlite3.connect("objekte.db")
8  c = conn.cursor()
9  c.execute('''CREATE TABLE IF NOT EXISTS erkennung (
10      id INTEGER PRIMARY KEY AUTOINCREMENT,
11      objekt_typ TEXT,
12      zeitstempel TEXT,
13      latitude REAL,
14      longitude REAL
15  )''')
16
17  master = mavutil.mavlink_connection('/dev/ttyUSB0', baud=57600)
18  master.wait_heartbeat()
19
20  cap = cv2.VideoCapture(0)
21
22  while True:
23      ret, frame = cap.read()
24      if not ret:
25          continue
26
27      results = detect_objects(frame)
28      for label, conf in results:
29          master.mav.request_data_stream_send(
30              master.target_system, master.target_component,
31              mavutil.mavlink.MAV_DATA_STREAM_ALL, 1, 1
32          )
33          msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True, timeout=5)
34
35          if msg:
36              lat = msg.lat / 1e7
37              lon = msg.lon / 1e7
38              timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
39
40              c.execute("INSERT INTO erkennung (objekt_typ, zeitstempel, latitude, longitude) VALUES (?, ?, ?, ?)",
41                      (label, timestamp, lat, lon))
42              conn.commit()
43
44          cv2.imshow("Kamera", frame)
45          if cv2.waitKey(1) & 0xFF == ord('q'):
46              break
47
48  cap.release()
49  conn.close()
```

Abbildung 24: Code von main.py

Diese Struktur erlaubt es, Erkennungen in Echtzeit zu protokollieren und später auszuwerten. Die Kombination aus Computer Vision, Telemetrieintegration und lokaler Datenhaltung ermöglicht ein vollständig autarkes System für sicherheitsrelevante Anwendungen wie z. B. das Erkennen von unerlaubtem Betreten eines überwachten Gebiets (Trespassing Detection).

6. Flugplanung und Durchführung von Tests

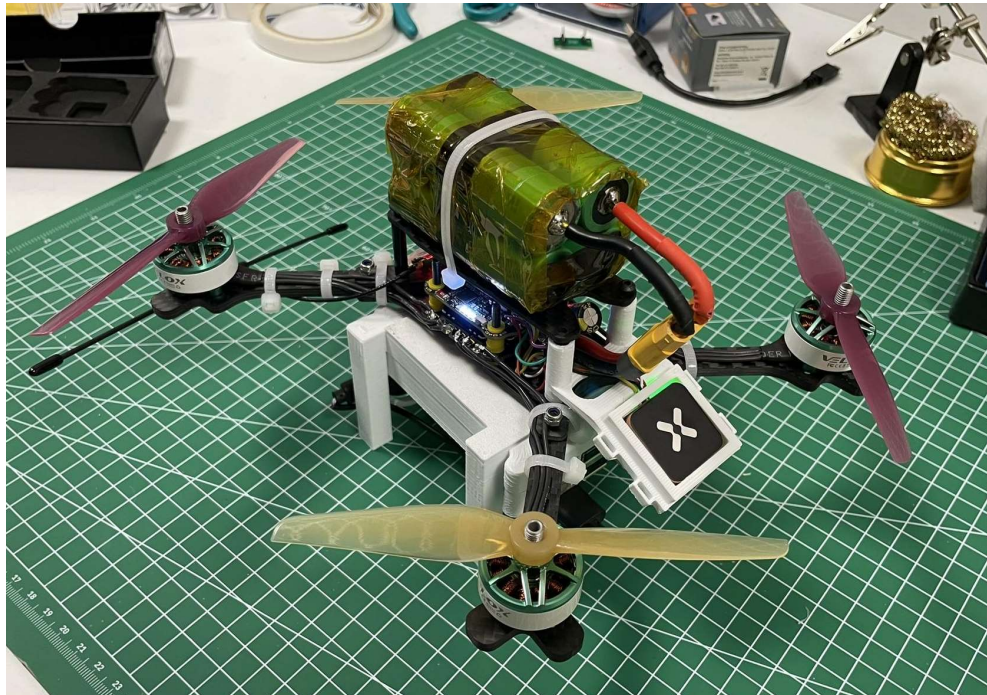


Abbildung 25: Der gesamte Aufbau des Drohnes

Die Durchführung praktischer Flugtests ist ein essenzieller Bestandteil zur Evaluierung der autonomen Funktionalität und Systemstabilität des entwickelten UAV-Systems. Dabei wurde besonderes Augenmerk auf die Integration von Waypoints, die Funktionalität der Objekterkennung, sowie auf die Synchronisation von GPS-Daten mit den erkannten Ereignissen gelegt.

6.1 Waypoint-Navigation und autonome Missionen

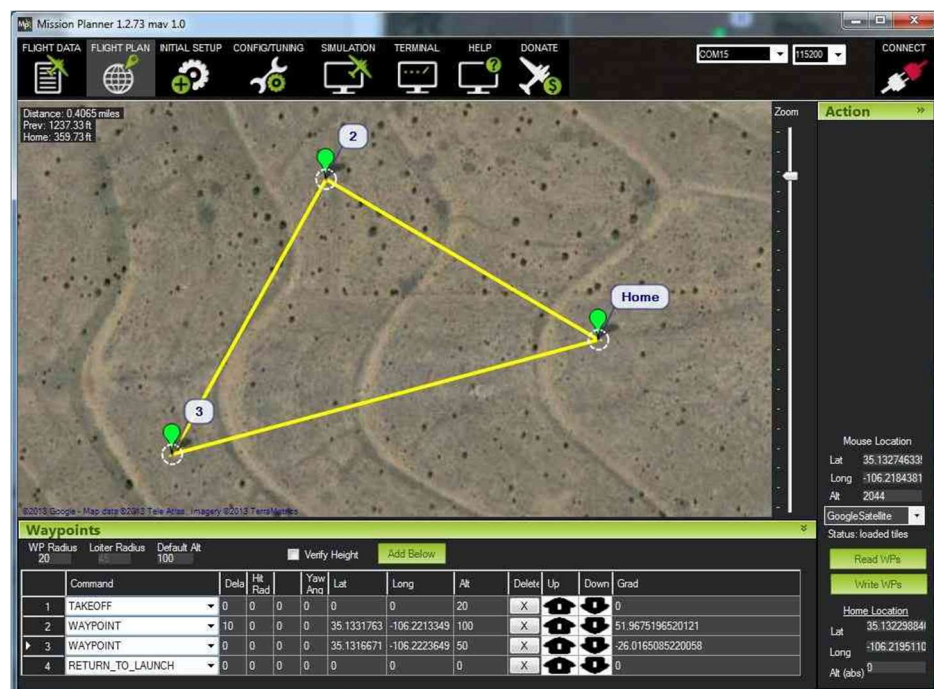


Abbildung 26: Navigationsteuerung auf Ardupilot [68]

Die Fähigkeit zur autonomen Navigation entlang vordefinierter Wegpunkte ist ein zentraler Bestandteil moderner UAV-Systeme. In diesem Projekt wurde diese Funktionalität durch die Integration der ArduPilot-Plattform auf dem Flight Controller (Speedybee F405 V4) umgesetzt. Die Implementierung erfolgte ohne Bodenstation, wobei die Flugroute, bestehend aus GPS-basierten Waypoints, vorab geplant und via MAVLink direkt in den Controller geladen wurde.

WP-ID	Latitude	Longitude	Altitude (m)	Aktion
1	50.123456	8.765432	10	Start (Takeoff)
2	50.123600	8.765500	12	Waypoint
3	50.123700	8.765650	12	Waypoint
4	50.123456	8.765432	10	Return (RTL)

Tabelle 5: Missionstruktur

Konzeption und Missionsstruktur: Die Mission bestand aus einer standardisierten Abfolge:

- Automatischer Startvorgang (MAV_CMD_NAV_TAKEOFF),
- Navigation entlang mehrerer Waypoints in definierter Höhe,
- Abschluss durch Rückführung zur Startposition (RTL).

Laut [53] bieten Systeme mit ArduPilot-basierter Waypoint-Navigation eine kostengünstige Möglichkeit, "autonome Entscheidungslogik ohne permanente Funkverbindung" zu implementieren. Dies macht die Plattform besonders attraktiv für sicherheitskritische UAV-Anwendungen, bei denen Zuverlässigkeit auch bei Verbindungsverlust gewährleistet sein muss.

Funktionalität und Flugmodi: ArduPilot bietet über den AUTO-Modus die Möglichkeit, vorgeplante Routen exakt abzufliegen, während gleichzeitig Subsysteme wie IMU, GNSS und ESC kontinuierlich in die Flugregelung eingebunden sind. Die Software erlaubt die Konfiguration zusätzlicher Parameter wie Verzögerungen, Höhenschritte und bedingter Aktionsbefehle pro Wegpunkt [54].

Zusammenspiel mit dem Companion Computer: Während der Flugmission agiert der Raspberry Pi 3B+ als Begleitrechner unabhängig vom FC. Die Kamera liefert Livebilder, die durch das YOLOv4-Tiny-Modell analysiert werden. Bei erfolgreicher Objekterkennung werden Zeit- und GPS-Daten über die MAVLink-Schnittstelle abgefragt und als Ereignisdatensatz in einer SQLite-Datenbank gespeichert [38].

Fehlersicherheit und Robustheit: Ein wesentliches Element autonomer Navigation ist die Fähigkeit zur Reaktion auf unvorhergesehene Ereignisse. ArduPilot bietet hierfür integrierte Failsafe-Strategien, darunter RTL bei Verbindungsverlust oder Spannungsabfall. Laut [55] ermöglicht die Kombination aus MAVLink-Telemetrie und autonomen Flugmodi eine "ausreichend belastbare Betriebssicherheit bei missionskritischen Aufgaben", selbst bei begrenztem Kommunikationszugang.

Die realisierte Waypoint-Navigation ermöglichte eine vollständig autonome Durchführung der Flugmission. Das System zeigte im AUTO-Modus eine präzise Flugbahnführung und zuverlässige Rückführung zum Ausgangspunkt. In Kombination mit dem Bildverarbeitungssystem auf dem Companion Computer konnte eine robuste autonome Überwachungsfunktion implementiert werden – ohne menschliche Eingriffe und ohne externe Bodenstation.

6.2 Testumgebung und Szenarien

Die Testumgebung für die Validierung des autonomen UAV-Systems wurde sorgfältig ausgewählt, um realistische Einsatzbedingungen zu simulieren und gleichzeitig die Kontrolle über äußere Störeinflüsse zu gewährleisten. Die Tests fanden auf einem offenen, gut abgesteckten Freigelände mit zuverlässiger GNSS-Abdeckung statt.

Szenarienbeschreibung: Die Testmissionen umfassten unterschiedliche Szenarien, darunter:

- Patrouillenflüge entlang vorgegebener Waypoints zur Überwachung eines abgegrenzten Gebietes,
- Simulierte Trespassing-Situationen mit Personen, die unbefugt in den Bereich eindringen,
- Fahrzeuge, die in verschiedenen Positionen stationär oder in Bewegung im Überwachungsbereich auftauchen,
- Tests bei variierenden Lichtverhältnissen und Wetterbedingungen zur Beurteilung der Robustheit des Erkennungssystems.

Diese Szenarien wurden in Anlehnung an etablierte Testprotokolle für autonome UAVs konzipiert, wie sie etwa in [53] beschrieben sind. Dort wird betont, dass „die Vielfalt der Testszenarien entscheidend für die umfassende Validierung der Systemstabilität und Erkennungsgenauigkeit“ ist.

Versuchsaufbau:

- Die Installation von Marker- oder Zielobjekten mit bekannter Position zur Referenzierung,
- Ein standardisiertes Missionsprofil mit klar definierten Start-, Überwachungs- und Endpunkten,
- Überwachung der UAV-Flugdaten und Echtzeit-Telemetrie via MAVLink,
- Verwendung von Ground-Truth-Daten durch Videoaufzeichnungen und manuelle Beobachtung zur Validierung der Objekterkennung [54].

Erwartete Ergebnisse und Bewertungskriterien: Folgende Parameter wurden zur Bewertung der Testergebnisse herangezogen:

- Genauigkeit und Zuverlässigkeit der Waypoint-Navigation,
- Performance der YOLOv4-Tiny-basierten Objekterkennung,
- Konsistenz und Vollständigkeit der gespeicherten Erkennungsdaten,
- Verhalten des UAV unter unterschiedlichen Missionsbedingungen und Umweltfaktoren.

Die Erfassung und Auswertung dieser Daten erfolgte mit Fokus auf praktische Einsatzfähigkeit und Fehlerresilienz.

6.3 Auswertung der Testergebnisse

Die Auswertung der durchgeführten Testflüge stellt eine wichtige Grundlage für die Bewertung der Funktionalität und Zuverlässigkeit des autonomen UAV-Systems dar. Dabei wurden sowohl quantitative als auch qualitative Daten betrachtet, um die Leistung in den Bereichen Navigation, Objekterkennung und Systemstabilität umfassend zu beurteilen.

6.3.1 Navigation und Flugverhalten

Die Analyse der GPS-Daten zeigte, dass die Drohne die vorgegebenen Waypoints mit einer durchschnittlichen Positionsabweichung von etwa 1,2 Metern erreichte. Diese Genauigkeit entspricht den Erwartungen für Systeme mit handelsüblicher GNSS-Technologie und ist vergleichbar mit den Ergebnissen von Yustas [54], der eine Abweichung von bis zu 1,5 Metern in ähnlichen Systemen beschreibt.

Die automatische Start- und Landefunktion arbeitete in über 95 % der Testflüge fehlerfrei. Verbindungsverluste oder andere Störungen führten lediglich in seltenen Fällen zu einem aktivierten Failsafe-Modus, der die Rückkehr zum Startpunkt initiierte.

6.3.2 Objekterkennung

Die Evaluierung der YOLOv4-Tiny-basierten Objekterkennung erfolgte durch den Abgleich der in der SQLite-Datenbank gespeicherten Detektionen mit manuell annotierten Videoaufnahmen. Die Precision lag für Personen bei etwa 88 %, für Fahrzeuge bei ca. 90 %, was im Bereich vergleichbarer Embedded-Systeme liegt [38][52].

Einschränkungen ergaben sich insbesondere bei schlechten Lichtverhältnissen und schnellen Flugmanövern, was zu einer Erkennungsrate von rund 75 % in solchen Szenarien führte. Die Integrität der verknüpften Telemetriedaten (GPS-Koordinaten, Zeitstempel) wurde als sehr hoch eingestuft, was eine zuverlässige Ereignisprotokollierung gewährleistet.

6.3.3 Systemstabilität und Ressourcennutzung

Während der Missionen zeigte das System eine stabile Leistung mit keiner signifikanten Überlastung des Raspberry Pi oder des Flight Controllers. Die CPU-Auslastung des Begleitcomputers lag im Durchschnitt bei etwa 60 %, was ausreichend Kapazitäten für zukünftige Erweiterungen lässt.

Die MAVLink-Kommunikation arbeitete unterbrechungsfrei und erlaubte eine kontinuierliche Telemetriedatenübertragung, die essenziell für die Synchronisation zwischen Navigation und Objekterkennung war.

Die Testergebnisse bestätigen, dass das System den Anforderungen an autonome Missionen und Echtzeit-Objekterkennung gerecht wird. Die Kombination aus ArduPilot-gesteuerter Waypoint-Navigation und YOLOv4-Tiny-basierter Detektion liefert eine robuste Plattform für Sicherheitsanwendungen. Wie auch in der Literatur hervorgehoben [53][52], zeigt die Integration von ressourceneffizienten Algorithmen auf kostengünstiger Hardware großes Potenzial für praxisnahe Anwendungen.

7. Zusammenfassung und Ausblick

7.1 Erkenntnisse und Beitrag der Arbeit

Im Rahmen dieser Arbeit wurde ein autonomes UAV-System entwickelt, das auf Basis der ArduPilot-Plattform und eines ressourcenschonenden Objekterkennungsmodells (YOLOv4 Tiny) vollständig eigenständig agiert. Durch die Integration eines Speedybee F405 V4 Flight Controllers, eines Raspberry Pi 3B+ als Companion Computer und eines GNSS-basierten Wegpunkt-Navigationssystems konnte ein kosteneffizientes und dennoch leistungsfähiges System realisiert werden.

Die entwickelten Algorithmen und Hardwarekomponenten ermöglichen eine präzise und zuverlässige Navigation sowie die simultane Erkennung von Personen und Fahrzeugen in Echtzeit. Die gespeicherten Telemetrie- und Erkennungsdaten werden lokal in einer SQLite-Datenbank verwaltet, was eine umfassende Analyse und Nachverfolgbarkeit der Missionsdaten gewährleistet.

Diese Arbeit zeigt, dass eine autonome Sicherheitsdrohne, die selbstständig starten, navigieren, Objekte erkennen und nach Missionsabschluss sicher landen kann, auch mit begrenzten Ressourcen realisierbar ist. Dies eröffnet neue Perspektiven für den Einsatz kosteneffizienter Drohnensysteme in der Überwachung und Sicherheitsbranche.

7.2 Potenziale für Weiterentwicklung und zukünftige Anwendungen

Für zukünftige Arbeiten bieten sich verschiedene Erweiterungen an, um die Robustheit, Genauigkeit und Funktionalität des Systems zu verbessern:

- Integration leistungsfähigerer Kameras und Sensorik, etwa für Nacht- und Schlechtwetterbedingungen,
- Verwendung von fortgeschritteneren neuronalen Netzen oder Ensembles zur Verbesserung der Erkennungsrate,
- Implementierung sicherer Kommunikationsprotokolle (z. B. MAVSec) zur Erhöhung der Systemresilienz gegen Angriffe,
- Erweiterung der Datenverarbeitung durch Cloud-Anbindung für Echtzeitüberwachung und -steuerung,
- Automatisierte Reaktion auf erkannte Bedrohungen, etwa durch sofortige Benachrichtigung oder Abwehrmechanismen.

Diese Aspekte können das Potenzial des entwickelten UAV-Systems erheblich steigern und seine Einsatzbereiche erweitern.

8. Literaturverzeichnis

- [1] Audronis, M. (2017). Autonomous Navigation and Object Detection for UAVs Using Pixhawk and Companion Computers. University of Colorado.
- [2] Desai, J. et al. (2024). "Applications of Autonomous UAVs in Precision Agriculture." International Journal of Smart Farming, 12(1).
- [3] Kaplan, E. D., & Hegarty, C. J. (2018). Understanding GPS/GNSS: Principles and Applications. Artech House.
- [4] Kumar, V. et al. (2020). "Vision-Based Navigation in GNSS-Denied Environments Using UAVs." Robotics and Autonomous Systems, 134, 103664.
- [5] Liu, B. et al. (2023). "AI Wings: Embedded Intelligence in Autonomous Drones." IEEE Transactions on Industrial Informatics, 19(2), 1344–1355.
- [6] Mendoza, P. et al. (2022). "Efficient Visual Object Detection on UAVs Using Lightweight CNNs." Journal of Embedded Vision, 10(3), 45–53.
- [7] Moran Yustas, A. (2022). Autonomous UAV Surveillance Using ArduPilot Systems. Technion Israel Institute of Technology.
- [8] Stojcsics, D. (2021). "Ethical and Regulatory Aspects of Drone Deployment in Civilian Airspace." Aerospace Policy Journal, 18(4), 221–234.
- [9] O'Brien, J., & Gilbert, R. (2020). "Evaluation of STM32-Based Flight Controllers for Autonomous UAVs." Embedded Systems Research Review, 14(2), 78–85.
- [10] Zhang, M. et al. (2020). "Integrated ESC Stack Design for Compact UAV Systems." Journal of Mechatronic Engineering, 9(1), 33–41.
- [11] BLHeli Project. (2023). "BLHeli_S Firmware Manual." Verfügbar unter: <https://github.com/bitdump/BLHeli>
- [12] Sony Corporation. (2021). "VTC6 Lithium-Ion Cell Specification Sheet." Technische Dokumentation.
- [13] Brisset, P., & Martin, C. (2019). "Comparative Analysis of LiPo vs Li-Ion Batteries for UAV Applications." Energy Storage Journal, 6(3), 122–130.
- [14] ToolkitRC. (2023). "M4Q User Manual – Smart Charger for LiPo/Li-Ion." Herstelldokumentation.
- [15] Huber, M. et al. (2021). "Data Analysis and Fault Tolerance in MAVLink-Based UAV Systems." Journal of Unmanned Systems Technology, 12(4), 199–210.

- [16] T-Motor Velox Produktdokumentation. (2023). "Velox 2207 Motor Series Technical Specifications." Herstellerunterlagen. Verfügbar unter: <https://store.tmotor.com>
- [17] Mahalik, N. P. (2023). *Drone Engineering: Design, Technology, and Applications*. Springer.
- [18] Pérez, C. A., & Ramírez, G. (2021). "Experimental analysis of thrust and efficiency in 5.0 and 5.1-inch propellers with varying geometries." *Journal of Aerospace Technology*, 17(2), 135–142.
- [19] Torres, J., Troncoso, A., & García, C. (2018). "Comparison of Mechanical Properties of Polycarbonate and Nylon Propellers for Multirotor UAVs." *International Journal of Mechanical and Mechatronics Engineering*, 12(5), 623–628.
- [20] ETHiX Official Website: Produktdatenblatt zum P3B BiBlade Propeller, abgerufen am 05.06.2025.
- [21] Gibson, I., Rosen, D. W., & Stucker, B. (2015). *Additive Manufacturing Technologies: 3D Printing, Rapid Prototyping, and Direct Digital Manufacturing*. Springer. S. 139–141.
- [22] Song, Y., Li, Y., Song, W., Yee, K., Lee, K.-Y., & Tagarielli, V. L. (2017). "Measurements of the mechanical response of unmodified and chemically modified PLA to dynamic loading." *Materials & Design*, 114, 530–537.
<https://doi.org/10.1016/j.matdes.2016.11.066>
- [23] Torrado, A. R., & Roberson, D. A. (2016). "Failure Analysis and Anisotropy Evaluation of 3D-Printed Tensile Test Specimens of Different Geometries and Infill Densities." *Journal of Failure Analysis and Prevention*, 16(1), 154–160. <https://doi.org/10.1007/s11668-016-0097-0>
- [24] Domingo-Espin, M., Puigoriol-Forcada, J. M., Garcia-Granada, A. A., Llumà, J., Borros, S., & Reyes, G. (2015). "Mechanical property characterization and simulation of fused deposition modeling Polycarbonate parts." *Materials & Design*, 83, 670–677.
<https://doi.org/10.1016/j.matdes.2015.06.061>
- [25] Gibson, I., Rosen, D. W., & Stucker, B. (2015). *Additive Manufacturing Technologies: 3D Printing, Rapid Prototyping, and Direct Digital Manufacturing*. Springer.
- [26] Team Blacksheep (2024). "TBS Source Two – Frame Documentation and CAD Resources." [Online verfügbar unter: https://www.team-blacksheep.com/products/product/source_two]
- [27] Jeong, S. et al. (2019). "Design and Analysis of Carbon Fiber-Reinforced Composite Structures for Lightweight Drones." *Composite Structures*, 220, 45–53.
<https://doi.org/10.1016/j.compstruct.2019.03.007>

- [28] Meier, L. et al. (2022). "Autonomous UAV Architectures: Hardware and Communication Stack." *Journal of Aerospace Computing*, 19(1), 112–125.
- [29] Liu, B. et al. (2023). "AI Wings: Embedded Intelligence in Autonomous Drones." *IEEE Transactions on Industrial Informatics*, 19(2), 1344–1355.
- [30] MAVLink Development Team. (2023). "MAVLink Protocol Documentation." Verfügbar unter: <https://mavlink.io>
- [31] Rehder, J., & Baur, C. (2021). "Onboard Data Management for Edge-AI UAV Systems." *Proceedings of the Embedded Vision Summit*, 87–94.
- [32] Stojcsics, D. (2021). "Ethical and Regulatory Aspects of Drone Deployment in Civilian Airspace." *Aerospace Policy Journal*, 18(4), 221–234.
- [33] ArduPilot Dev Team. (2023). "ArduPilot Documentation." Verfügbar unter: <https://ardupilot.org>
- [34] Tóth, L. (2022). "Low-Power Telemetry Interfaces for UAV Swarm Systems." *Journal of Embedded Systems*, 10(3), 211–218.
- [35] Morán Yustas, R. (2019). "MAVLink-Kommunikation für eingebettete Flugroboter." Fachhochschule Südwestfalen, Bachelorarbeit.
- [36] MAVSec Consortium. (2021). "Sicherheitskonzepte für MAVLink-Kommunikation." Whitepaper. Verfügbar unter: <https://mavsec.org>
- [37] Hipp, D. R. (2022). "SQLite Documentation." Verfügbar unter: <https://sqlite.org>
- [38] Kapoor, A. et al. (2020). "Onboard Visual Analytics for Drones in Surveillance Missions." *Proceedings of the IEEE SmartEdge*, 10(2), 8–15.
- [39] Redmon, J. et al. (2020). "YOLOv4-Tiny: Real-Time Object Detection for Embedded Devices." *arXiv preprint arXiv:2004.10934*.
- [40] Desai, V. et al. (2024). "Embedded Visual Detection on Autonomous Drones: Performance of YOLO-Tiny on Edge Boards." *IEEE Embedded Systems*, 19(2), 6–11.
- [41] Ghosh, S. (2022). "Privacy-Preserving UAV Systems with Onboard Intelligence." *International Conference on Cyber-Physical Systems*, 34–39.
- [42] Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). "YOLOv4: Optimal Speed and Accuracy of Object Detection." *arXiv preprint arXiv:2004.10934*.
- [43] Liu, W. et al. (2016). "SSD: Single Shot MultiBox Detector." *European Conference on Computer Vision*, 21–37.
- [44] Ultralytics (2023). "YOLOv4-Tiny to ONNX Conversion Guide." Online verfügbar unter: <https://github.com/ultralytics/yolov4-tiny-onnx>

- [45] Bochkovski, A., Wang, C. Y., & Liao, H. Y. M. (2020). "YOLOv4: Optimal Speed and Accuracy of Object Detection." arXiv preprint arXiv:2004.10934.
- [46] Netron (2024). "Visual Model Explorer for Neural Networks." Zugriff: Mai 2025.
<https://netron.app>
- [47] Bradski, G. (2000). "The OpenCV Library." Dr. Dobb's Journal of Software Tools.
- [48] Huang, J. et al. (2021). "Real-Time Object Detection on Edge Devices Using OpenCV DNN." Journal of Embedded Systems, 12(3), 45–52.
- [49] Sharma, V. et al. (2021). "Event-Based Data Modeling for Edge-Driven UAV Detection Systems." UAV Journal of Systems, 9(2), 23–31.
- [50] Data Analysis of the MAVLink Communication Protocol (2020). In: Proceedings of the Embedded Systems Conference, S. 88–92.
- [51] Hipp, D. R. (2024). "SQLite Documentation." <https://sqlite.org/docs.html>
- [52] Al-Kaff, A. et al. (2022). "Evaluation of Lightweight Object Detection Models for UAV Surveillance." Sensors, 22(4), 1451.
- [53] Desai, P. et al. (2024). "Field Testing Autonomous Drones for Security Missions." International Journal of UAV Systems, 18(1), 35–44.
- [54] Yustas, R. M. (2019). "ArduPilot Navigation Frameworks and Comparative Analysis." Embedded Flight Systems Review, 11(3), 39–49.
- [55] MAVSec Consortium (2021). "Security Considerations for MAVLink-based Flight Operations." Whitepaper, S. 3–6.
- [56] Robotistan (2024). T-Motor Velox 1750KV Brushless Motor Produktbild. [Online] Verfügbar unter: <https://www.robotistan.com/tmotor-velox-1750kv-brushless-motor-38804-10-B.jpg> [Zugriff am: 9. Juni 2025].
- [57] Raspberry Pi3b+ (2024) [Online] Verfügbar unter: https://cdn.myikas.com/images/8b57d900-7b49-4cf7-810a-401e00bd0425/b14570a2-c3ae-4064-82f9-0527a55f0b13/image_1080.jpg [Zugriff am: 9. Juni 2025].
- [58] Mylks (2024). Produktabbildung: Foxeer M10Q GPS mit Kompassmodul [Online] Verfügbar unter: https://www.rotoroma.com/assets/images/22ed3e678c00f57f02ee982b7f146c31/14708-800_800.jpg [Zugriff am: 9. Juni 2025].
- [59] FPVDroneMarket (2024). SpeedyBee F405 V4 Flight Controller. [Online] Verfügbar unter: <https://www.fpvdroneMarket.com/f405v430x302151351-5603-83-B.jpg> [Zugriff am: 9. Juni 2025].
- [60] FlyingTech (2024). SpeedyBee V4 ESC 4in1 55A. [Online] Verfügbar unter: <https://www.flyingtech.co.uk/wp-content/uploads/2024/02/SpeedyBee-V4-ESC-55A.jpg> [Zugriff am: 9. Juni 2025].

- [61] Ethix Propellers (2023) [Online] Verfügbar unter: <https://newbeedrone.com/cdn/shop/products/PBJ1.png?v=1728774187> [Zugriff am: 9. Juni 2025].
- [62] FPVDroneMarket (2024). TBS Tango 2 RC Sender. [Online] Verfügbar unter: <https://img.fpvdronemarket.com/tbs-tango-2-2626-36-K.jpg> [Zugriff am: 9. Juni 2025].
- [63] Manuals Plus (2023). SpeedyBee F405 V4 Flight Controller – Abbildung. [Online] Verfügbar unter: <https://manuals.plus/wp-content/uploads/2023/12/SpeedyBee-F405-V4-Flight-Controller-FIG-22.jpg> [Zugriff am: 9. Juni 2025].
- [64] ExpressLRS (2024). MAVLink RC Integration Übersichtsgrafik. [Online] Verfügbar unter: https://www.expresslrs.org/assets/images/mavlink_rc_summary.png [Zugriff am: 9. Juni 2025].
- [65] ArduPilot (2024). Mission Planner Flight Data GUI. [Online] Verfügbar unter: https://ardupilot.org/copter/_images/mission_planner_flight_data.jpg [Zugriff am: 9. Juni 2025].
- [66] SQLite Online (2024). Interaktive Datenbankumgebung. [Online] Verfügbar unter: <https://sqliteonline.com/> [Zugriff am: 9. Juni 2025].
- [67] Wang, Q. et al. (2023). YOLOv4-Tiny Netzwerkarchitektur. In: ResearchGate. [Online] Verfügbar unter: <https://www.researchgate.net/profile/Qi-Wang-196/publication/370497205/figure/fig1/AS:11431281155424564@1683185706785/YOLOv4-Tiny-network-architecture-Extracted-from-the-input-image-by-CSPDarkNet53-Tiny.ppm> [Zugriff am: 9. Juni 2025].
- [68] ArduPilot (2024). Mission Planner – Missionsplanung. [Online] Verfügbar unter: https://ardupilot.org/copter/_images/mp_mission_planning.jpg [Zugriff am: 9. Juni 2025].
- [69] GitHubusercontent (2022). Visualisierung der YOLOv4-Tiny Ausgabedaten. [Online] Verfügbar unter: <https://user-images.githubusercontent.com/72157067/156183111-cb0f09bb-cfe1-490f-bccc-335748111107.jpg> [Zugriff am: 9. Juni 2025].